دانشگاه تهران

دانشکدگان فارابی

دانشکده مهندسی

گروه مهندسی کامپیوتر

طراحی و پیاده‌سازی دستیار هوشمند بالینی مبتنی بر تلفیق پرونده بیمار و دانش پزشکی معتبر

Clinical AI Copilot

نگارش

ایمان کربلائی

استاد راهنما

دکتر کاظم فولادی قلعه

گزارش پروژه برای دریافت درجه کارشناسی در رشته مهندسی کامپیوتر

سال تحصیلی ۱۴۰۵–۱۴۰۶

ماه و سال ارائه: مهر ۱۴۰۵


بسم الله الرحمن الرحيم

مشخصات جلسه داوری

نام دانشجو: ایمان کربلائی

شماره دانشجویی: ۲۲۰۷۰۱۰۸۷

عنوان پروژه: طراحی و پیاده‌سازی دستیار هوشمند بالینی (Clinical AI Copilot) مبتنی بر تلفیق پرونده بیمار و دانش پزشکی معتبر

استاد راهنما: دکتر کاظم فولادی قلعه

تاریخ جلسه:

اعضای هیئت داوری:

نتیجه و نمره:

محل امضاها: .....

چکیده

این پروژه یک دستیار بالینی تحت وب است که اطلاعات پرونده بیمار، اسناد، یادداشت صوتی و گفت‌وگوی مبتنی بر مدل زبانی را در یک محیط فارسی کنار هم قرار می‌دهد. گزارش بر بخشی تمرکز دارد که در تقسیم کار پروژه به ایمان کربلائی سپرده شده است: معماری فرانت‌اند و طراحی گردش کار پزشک. رابط با Next.js، React و TypeScript ساخته شده و داده‌های هر بخش را از API دریافت می‌کند. جداسازی اطلاعات ثبت‌شده پرونده از پاسخ مدل، نمایش وضعیت عملیات و امکان بازبینی متن پیش از ذخیره، از تصمیم‌های اصلی این پیاده‌سازی‌اند.

در بررسی نسخه فعلی، مسیر ورود، مدیریت بیمار، خلاصه پرونده، تایم‌لاین، اسناد، یادداشت صوتی، گفت‌وگوی بالینی و ایمنی دارو دنبال شد. بعضی قابلیت‌های سرور هنوز در رابط کامل نشده‌اند و مسیر آزمایشی Agent نیز جدا از گفت‌وگوی اصلی باقی مانده است. ساخت production و بررسی TypeScript موفق بود، اما ESLint هفت خطا و دو هشدار گزارش کرد. آزمون مرورگری با داده کاملاً ساختگی انجام شد و شامل ارزیابی بالینی، OCR واقعی، رونویسی صوت یا سنجش کیفیت پاسخ مدل نبود.

کلیدواژه‌ها: رابط کاربری فارسی، Next.js، پرونده بیمار، مستندسازی صوتی، گفت‌وگوی بالینی، شواهد داخلی، کاربردپذیری، تصمیم‌یار بالینی.

فهرست مطالب

فصل ۱ مقدمه و تعریف مسئله ۱

۱ـ۱ مسئله از دید رابط کاربری ۱

۱ـ۲ هدف و دامنه نمونه دانشگاهی ۱

۱ـ۳ مسئولیت ایمان و مرز کار مشترک ۱

۱ـ۴ تبدیل نیاز به کنش قابل اجرا ۲

۱ـ۵ نیازهای غیرعملکردی و معیار پذیرش ۲

فصل ۲ نیازمندی‌ها و مبانی طراحی رابط ۵

۲ـ۱ جست‌وجو و تشکیل پرونده ۵

۲ـ۲ اسناد و بازبینی اطلاعات ۵

۲ـ۳ ثبت صوت و تأیید انسانی ۵

۲ـ۴ سؤال، شواهد و هشدار دارویی ۶

۲ـ۵ جهت متن و بومی‌سازی ۶

فصل ۳ معماری و ساختار فرانت‌اند ۹

۳ـ۱ ساختار مسیرها و مسئولیت پوشه‌ها ۹

۳ـ۲ فضای کار پرونده ۱۰

۳ـ۳ هویت بیمار به‌عنوان زمینه ثابت ۱۰

۳ـ۴ تب‌ها و آدرس‌پذیری ۱۰

۳ـ۵ اجزای عمومی و قرارداد بازاستفاده ۱۱

فصل ۴ قرارداد داده و ارتباط با API ۱۳

۴ـ۱ یک مرز مشترک برای درخواست‌ها ۱۳

۴ـ۲ خواندن اولیه و ثبت تغییر ۱۳

۴ـ۳ خطا به‌عنوان بخشی از قرارداد ۱۴

۴ـ۴ جدول قراردادهای اصلی ۱۵

۴ـ۵ مدیریت وضعیت فرم در مرز درخواست ۱۶

فصل ۵ ورود و مدیریت بیماران ۱۷

۵ـ۱ مشاهده مستقیم ورود و ثبت‌نام ۱۷

۵ـ۲ مرز اعتماد بین رابط و سرور ۱۸

۵ـ۳ ورود و ثبت‌نام در رابط فعلی ۱۸
۵ـ۴ توکن و نگهداری نشست ۱۹
۵ـ۵ چرخه ورود به فهرست ۱۹
۵ـ۶ جست‌وجو و حفظ زمینه پیمایش ۲۰
۵ـ۷ فرم ایجاد و ویرایش بیمار ۲۰
فصل ۶ پرونده بیمار و مدیریت اسناد ۲۳
۶ـ۱ خلاصه به‌عنوان نمای منتخب داده ۲۳
۶ـ۲ اعلان حساسیت و رابطه با ثبت داده ۲۴
۶ـ۳ داروها و شرایط فعال ۲۴
۶ـ۴ آزمایش‌های اخیر و محدودیت برداشت ۲۴
۶ـ۵ ساخت و نمایش روند زمانی ۲۵
۶ـ۶ نقش سند در پرونده ۲۶
۶ـ۷ انتخاب فایل و اطلاعات کمکی ۲۷
۶ـ۸ وضعیت استخراج و زمان اطلاع کاربر ۲۹
فصل ۷ گفت‌وگوی بالینی و نمایش شواهد ۳۱
۷ـ۱ نقش پنل دستیار در فضای پرونده ۳۱
۷ـ۲ ایجاد، انتخاب و حذف گفتگو ۳۲
۷ـ۳ ورودی، ارسال و حفظ متن ۳۳
۷ـ۴ تاریخچه گفتگو و زمینه بیمار ۳۳
۷ـ۵ شواهد داخلی و مراجعه به اصل ۳۴
۷ـ۶ تصمیم‌های رابط برای قابل پیگیری ماندن پاسخ ۳۴
فصل ۸ یادداشت صوتی و بررسی ایمنی دارو ۳۵
۸ـ۱ دو سناریو در یک گردش کار ۳۵
۸ـ۲ انتخاب حالت و مدیریت ورودی ۳۵
۸ـ۳ آغاز ضبط و سازگاری نوع رسانه ۳۷
۸ـ۴ زمان، توقف و آزادسازی میکروفون ۳۷
۸ـ۵ ارسال و وضعیت پردازش ۳۸
۸ـ۶ متن خام، نسخه مشتق‌شده و فرم بازبینی ۳۹
۸ـ۷ هدف و تفاوت با تجویز ۳۹

۸ـ۸ اجرای تحلیل و استقلال وضعیت‌ها ۴۰
فصل ۹ رابط فارسی و طراحی واکنش‌گرا ۴۳
۹ـ۱ مشاهده محدود چیدمان موبایل ۴۳
۹ـ۲ وضوح کنش و پیشگیری از خطا ۴۵
۹ـ۳ اعلام وضعیت و بازیابی شکست ۴۶
۹ـ۴ صفحه‌کلید و تمرکز دیداری ۴۶
فصل ۱۰ آزمون و تحویل سامانه ۴۷
۱۰ـ۱ سطوح متفاوت راستی‌آزمایی ۴۷
۱۰ـ۲ نتیجه اجرای جاری فرانت‌اند ۴۷
۱۰ـ۳ سناریوهای یکپارچه‌سازی ۴۸
۱۰ـ۴ توپولوژی سرویس و فایل اصلی ۴۹
۱۰ـ۵ پروکسی، زمان انتظار و حد حجم ۵۰
فصل ۱۱ ارزیابی و جمع‌بندی ۵۳
۱۱ـ۱ نتیجه کار من در نسخه موجود ۵۳
۱۱ـ۲ محدودیت ارزیابی انجام‌شده ۵۴
۱۱ـ۳ اصلاح‌های ضروری رابط ۵۴
۱۱ـ۴ آنچه در مرورگر اجرا کردم ۵۵
۱۱ـ۵ ادامه آزمون یکپارچه ۵۵
۱۱ـ۶ جمع‌بندی فنی ۵۵
۱۱ـ۷ حدود نتیجه ۵۵
۱۱ـ۸ مسیر ادامه ۵۶

فهرست شکل‌ها

شکل ۱ ـ معماری ارائه و مرزهای ارتباط در فرانت‌اند ۹
شکل ۲ ـ چرخه ثبت تغییر و بازآوری داده در رابط ۱۴
شکل ۳ ـ کارت ورود پزشک در نمای دسکتاپ ۱۷
شکل ۴ ـ حالت خالی فهرست بیماران با مسیر آغاز ثبت ۱۹

شکل ۵ ـ فهرست واقعی با یک بیمار کاملاً ساختگی ۱۹
شکل ۶ ـ پنجره‌ی واقعی ایجاد بیمار و فیلدهای هویتی ۲۱
شکل ۷ ـ کارت‌های خلاصه‌ی پرونده در فضای کار واقعی ۲۳
شکل ۸ ـ رویداد تایم‌لاین پس از ثبت داروی نمایشی ۲۵
شکل ۹ ـ منشأ داده‌های نمای خلاصه و تایم‌لاین ۲۶
شکل ۱۰ ـ تب اسناد و یادداشت‌ها با سه نوع رکورد ۲۶
شکل ۱۱ ـ چرخه بارگذاری سند و نمایش وضعیت استخراج ۲۷
شکل ۱۲ ـ پنجره‌ی انتخاب فایل و فراداده‌ی سند ۲۸
شکل ۱۳ ـ حالت خالی پنل گفت‌وگو و پرسش‌های پیشنهادی ۳۱
شکل ۱۴ ـ گردش پیام پایدار و مسیر جداگانه آزمایشی Agent ۳۲
شکل ۱۵ ـ پنجره‌ی واقعی ضبط یا انتخاب یادداشت صوتی ۳۶
شکل ۱۶ ـ ثبت صوت، رونویسی و تأیید یادداشت بالینی ۳۸
شکل ۱۷ ـ ردیف داروی نمایشی پس از ثبت از فرم برنامه ۳۹
شکل ۱۸ ـ کنترل‌های حالت ایمنی دارو در پنل گفت‌وگو ۴۰
شکل ۱۹ ـ استقلال تحلیل ایمنی از ثبت رکورد دارویی ۴۱
شکل ۲۰ ـ بخش آغازین فضای پرونده در نمای موبایل ۴۴
شکل ۲۱ ـ کارت‌های خلاصه‌ی پرونده در چیدمان تک‌ستونی موبایل ۴۵
شکل ۲۲ ـ توپولوژی پیکربندی تحویل و مسیر فایل ۵۰

فهرست جدول‌ها

جدول ۱ ـ قراردادهای اصلی مورد استفاده رابط ۱۵
جدول ۲ ـ وضعیت استخراج و رفتار قابل مشاهده رابط ۲۹
جدول ۳ ـ تفاوت مسیر قواعد و تحلیل AI در رابط دارو ۴۰
جدول ۴ ـ نتایج بررسی‌های اجراشده در نسخه جاری ۴۷
جدول ۵ ـ سناریوهای یکپارچه‌سازی و سطح شاهد ۴۸
جدول ۶ ـ تنظیمات تحویل مؤثر بر رفتار رابط ۵۰
جدول ۷ ـ تطبیق وظایف ایمان با امکانات جاری ۵۳

فصل ۱ مقدمه و تعریف مسئله

۱ـ۱ مسئله از دید رابط کاربری

در این پروژه روی رابطی کار کردم که اطلاعات پراکنده بیمار را در یک فضای واحد در اختیار پزشک می‌گذارد. مسئله فقط نمایش داده‌های API نبود. کاربر باید ابتدا بیمار درست را پیدا کند، بداند هر داده از کجا آمده است و نتیجه عملیات‌هایی مانند ثبت سند یا ارسال پرسش را همان‌جا ببیند. اگر ارتباط با سرور قطع شود نیز نباید متن واردشده یا مسیر کار او از بین برود. به همین دلیل، طراحی رابط را بخشی از گردش کار سامانه در نظر گرفتم و اجزای آن را بر اساس کارهایی که پزشک در پرونده انجام می‌دهد سازمان دادم.

همه اطلاعات پرونده ارزش و منشأ یکسانی ندارند. مشخصات بیمار و داروها مستقیماً در پایگاه داده ثبت می‌شوند، متن سند از یک مرحله پردازش عبور می‌کند و پاسخ دستیار نیز خروجی مدل زبانی است. برای اینکه این موارد با هم اشتباه نشوند، پرونده و پنل دستیار در دو ناحیه جدا قرار گرفته‌اند و محتوای تولیدی نشان و رنگ زمینه مخصوص دارد. این جداسازی صحت پاسخ را تضمین نمی‌کند، ولی به کاربر کمک می‌کند داده ثبت‌شده را از توضیح تولیدشده تشخیص دهد. در فصل‌های بعد توضیح می‌دهم این تصمیم در اجزای مختلف رابط چگونه پیاده شده است.

۱ـ۲ هدف و دامنه نمونه دانشگاهی

هدف نمونه اولیه این بود که چند قابلیت آماده در قالب یک وب‌اپلیکیشن منسجم کنار هم قرار گیرند. آموزش مدل زبانی، ساخت مدل تشخیص تصویر، تجویز خودکار و اتصال عملیاتی به سامانه بیمارستانی در برنامه این نسخه نبود. بنابراین در طراحی رابط، هوش مصنوعی را یک سرویس کمکی در نظر گرفتم. کاربر پیش از ذخیره رونوشت امکان بازبینی دارد و نتیجه بررسی دارو نیز جدا از دکمه ذخیره نمایش داده می‌شود. این دو نمونه نشان می‌دهند تصمیم نهایی همچنان در اختیار کاربر است.

دامنه رابط شامل صفحه معرفی، ورود و ثبت‌نام، فهرست بیماران، فضای کار پرونده، اسناد، یادداشت صوتی، خلاصه، تایم‌لاین و گفت‌وگوی بالینی است. در زمان بررسی نسخه نهایی متوجه شدم بعضی هدف‌های پروپوزال هنوز کامل نشده‌اند. متن استخراج‌شده سند قابل مشاهده است، اما ویرایشگر OCR در پنجره فعلی وجود ندارد. اطلاعات آزمایش نیز در پرونده نمایش داده می‌شود، ولی مسیر استخراج خودکار همه آزمایش‌ها از سند در رابط کامل نشده است. در ادامه، امکانات موجود و بخش‌های باقی‌مانده را جدا از هم توضیح می‌دهم.

۱ـ۳ مسئولیت ایمان و مرز کار مشترک

طبق بخش ۲.۲ پروپوزال، مسئولیت من «Frontend و گردش کار بالینی» بوده است. معماری فرانت‌اند، اجزای قابل استفاده مجدد، داشبورد بیمار، رابط اسناد، ثبت و مرور صوت، تایم‌لاین، خلاصه، چت و نمایش هشدارها در

همین بخش قرار می‌گیرند. عرفان کربلائی مسئولیت اصلی بک‌اند، پایگاه داده، پردازش سمت سرور اسناد و صوت، ساخت زمینه بیمار و اتصال مدل را بر عهده داشته است. طراحی معماری کلان، قرارداد API، یکپارچه‌سازی، آزمون سرتاسری و استقرار نیز کار مشترک تعریف شده بود. به همین دلیل در این متن روی رابط کاربری تمرکز می‌کنم و برای توضیح جریان کامل، فقط در موارد لازم به سرور اشاره می‌کنم.

مخزن پروژه با یک حساب مشترک یا حساب عرفان نیز تغییراتی در بخش فرانت‌اند دارد؛ در نتیجه از روی نام commit نمی‌توان سهم هر نفر را خط‌به‌خط جدا کرد. من در این متن مسئولیت تعیین‌شده خودم و رفتار نسخه نهایی تیم را توضیح می‌دهم و بخش‌های مشترک را نیز با همین عنوان مشخص می‌کنم. هرجا اجرای یک قابلیت را خودم در محیط آزمایشی مشاهده کرده‌ام، نتیجه همان اجرا را می‌نویسم و درباره مسیرهایی که اجرا نشده‌اند فقط به ساختار کد اشاره می‌کنم.

۱ـ۴ تبدیل نیاز به کنش قابل اجرا

برای طراحی مسیر «مرور پرونده»، آن را به چند کار کوچک‌تر تقسیم کردم: یافتن بیمار، کنترل هویت، دیدن وضعیت فعلی، رفتن به سوابق و تشخیص منشأ اطلاعات. فهرست بیماران نقطه شروع این مسیر است. پس از ورود به پرونده، نوار هویتی هنگام جابه‌جایی میان تب‌ها در بالای صفحه می‌ماند و نام بیمار در پنل چت نیز تکرار می‌شود. هدف این بود که کاربر برای تشخیص بیمار جاری فقط به نشانی مرورگر وابسته نباشد. میزان اثر این طراحی بر خطای انتخاب بیمار در این پروژه اندازه‌گیری نشده است.

در این سامانه، نام یک دکمه به‌تنهایی رفتار مورد انتظار را مشخص نمی‌کند. هنگام ثبت یادداشت باید معلوم باشد متن از رونوشت خام آمده یا از نسخه پاک‌سازی‌شده و کاربر چه زمانی آن را تأیید کرده است. نتیجه بررسی دارو باید به‌عنوان تحلیل کمکی دیده شود و مشاهده سند نیز باید راه بازگشت به فایل اصلی را حفظ کند. این موارد در پروپوزال با مفهوم پیگیری منشأ داده و بازبینی انسانی مطرح شده بودند. در نسخه فعلی، بخشی از این مسیرها اجرا شده است؛ برای نمونه لینک فایل وجود دارد، ولی دسترسی آن با همه حالت‌های احراز هویت آزموده نشده است.

۱ـ۵ نیازهای غیرعملکردی و معیار پذیرش

برای ارزیابی رابط، پنج موضوع را در نظر گرفتم: نمایش درست فارسی، پیام خطای قابل فهم، حفظ ورودی کاربر، چیدمان در اندازه‌های مختلف و کنترل دسترسی. در بررسی RTL، ترتیب عبارت‌های فارسی و شناسه‌های لاتین را دیدم؛ برای خطا، مسیر ادامه کار را بررسی کردم؛ و برای نمایش موبایل، صفحه را در عرض محدود باز کردم. موفق‌شدن build فقط سازگاری ساخت را نشان می‌دهد و جای این بررسی‌های دیداری و رفتاری را نمی‌گیرد.

سناریوهای پذیرش را با داده ساختگی تعریف کردم. هر سناریو یک پیش‌شرط، یک کنش و نتیجه مورد انتظار دارد؛ برای مثال فرم بیمار بدون نام ارسال نشود، خطای API ورودی کاربر را پاک نکند و رونوشت پیش از

تأیید به یادداشت نهایی تبدیل نشود. اجرای کامل همه این سناریوها انجام نشد. فصل دهم موارد اجراشده و موارد باقی‌مانده را جداگانه فهرست می‌کند.

فصل ۲ نیازمندی‌ها و مبانی طراحی رابط

۲ـ۱ جست‌وجو و تشکیل پرونده

در صفحه بیماران سه کار اصلی در دسترس است: جست‌وجو، حرکت میان صفحه‌های نتیجه و ایجاد پرونده. عبارت جست‌وجو در پارامتر q نشانی صفحه نگه‌داری می‌شود و پس از حذف فاصله‌های اضافی، همراه limit و offset به سرور می‌رود. این انتخاب باعث شد نتیجه جست‌وجو با بازکردن پیوند یا استفاده از دکمه بازگشت مرورگر حفظ شود. برای فهرست خالی و جست‌وجوی بدون نتیجه نیز دو پیام جدا گذاشته شده است؛ در حالت اول کاربر به ساخت بیمار هدایت می‌شود و در حالت دوم می‌تواند عبارت جست‌وجو را پاک کند.

فرم ایجاد پرونده نام و نام خانوادگی را اجباری می‌گیرد و کد ملی، تاریخ تولد، جنسیت، تلفن و نشانی اختیاری‌اند. کد ملی را رشته نگه داشتم تا صفر ابتدای آن حذف نشود. inputMode عددی و pattern ده‌رقمی ورود را ساده‌تر می‌کنند، اما کنترل یکتایی در سرور انجام می‌شود. کنترل تاریخ، مقدار میلادی استاندارد API را می‌فرستد و هنگام نمایش با قالب فارسی دیده می‌شود؛ بنابراین فارسی‌بودن صفحه به معنای استفاده از تقویم جلالی در خود کنترل date نیست.

۲ـ۲ اسناد و بازبینی اطلاعات

گردش سند از انتخاب فایل و تعیین عنوان یا نوع شروع می‌شود. پس از ارسال، کاربر وضعیت پردازش را در فهرست می‌بیند و در پایان می‌تواند متن استخراج‌شده یا فایل اصلی را باز کند. ویژگی accept فقط انتخاب فایل را در مرورگر محدود می‌کند و کنترل نهایی نوع و اندازه در سرور است. موفقیت بارگذاری نیز با پایان استخراج متن یکی نیست؛ به همین دلیل وضعیت‌های pending، processing و completed جدا نمایش داده می‌شوند. در ادامه بهتر است زمان آخرین به‌روزرسانی و یک کنش روشن برای دریافت دوباره وضعیت نیز به رابط اضافه شود.

پروپوزال علاوه بر مشاهده، اصلاح داده استخراج‌شده را هم پیش‌بینی کرده بود. در نسخه فعلی، ViewExtractedTextButton متن را در یک بلوک pre باز می‌کند و امکان ذخیره نسخه ویرایش‌شده ندارد. در نتیجه این قسمت فعلاً فقط برای بازبینی است. آیکون مداد انتخاب مناسبی برای این رفتار نیست و بهتر است با آیکون مشاهده جایگزین شود. تبدیل خروجی OCR به رکوردهای ساختاریافته آزمایش نیز به فرم تأیید جداگانه نیاز دارد؛ چنین فرمی در رابط فعلی وجود ندارد.

۲ـ۳ ثبت صوت و تأیید انسانی

برای صوت دو حالت در نظر گرفته شده است: یادداشت تک‌نفره پزشک و مکالمه پزشک و بیمار. کاربر نوع گفتگو را پیش از ارسال انتخاب می‌کند؛ این انتخاب فقط نوع پردازش را به سرور می‌گوید و نقش گویندگان را

به‌صورت خودکار تأیید نمی‌کند. فایل پس از توقف ضبط ارسال می‌شود و رونویسی در بک‌اند انجام می‌گیرد، پس این بخش یک دستیار صوتی زنده نیست. امکان انتخاب فایل آماده نیز برای زمانی اضافه شده که میکروفون در دسترس نیست یا مرورگر از ضبط پشتیبانی نمی‌کند.

رونوشت به‌طور خودکار وارد یادداشت بالینی نمی‌شود. کاربر ابتدا متن و عنوان را در پنجره بازبینی می‌بیند و با دکمه «ثبت در پرونده» آن را ذخیره می‌کند. متن خام و نسخه پاک‌سازی‌شده اکنون در دو پنجره جدا باز می‌شوند و فرم ذخیره با نسخه پاک‌سازی‌شده پر می‌شود. برای کاهش اشتباه، بهتر است منشأ متن پیش‌فرض و تفاوت آن با رونوشت خام در همان فرم نمایش داده شود. در این پروژه دقت رونوشت یا میزان تغییر معنا بین دو نسخه اندازه‌گیری نشد.

۲ـ۴ سؤال، شواهد و هشدار دارویی

در پنل چت، پرسش همیشه همراه شناسه بیمار جاری ارسال می‌شود. هنگام انتظار یک پیام وضعیت دیده می‌شود و خطای سرویس در همان پنل باقی می‌ماند. گفت‌وگوها در سرور ذخیره می‌شوند و بعداً قابل بازکردن‌اند، اما زمینه بیمار برای هر پیام دوباره از داده روز پرونده ساخته می‌شود. بنابراین ممکن است پاسخ قدیمی بر نسخه‌ای از پرونده تکیه داشته باشد که بعداً تغییر کرده است. زمان پیام و شواهد داخلی نمایش داده می‌شوند، ولی نسخه کامل زمینه آن لحظه ذخیره نمی‌شود.

بررسی ایمنی دارو یک کنش اختیاری و جدا از ذخیره است. نتیجه در چهار بخش تداخل‌ها، عوامل مربوط به بیمار، اطلاعات ناقص و جمع‌بندی نمایش داده می‌شود. فرم فعلی endpoint تحلیل AI را فراخوانی می‌کند و از مسیر قواعد بک‌اند استفاده نمی‌کند؛ در نتیجه خروجی آن با هشدار قطعی مبتنی بر قواعد یکسان نیست. مستقل‌بودن دکمه ذخیره نیز یعنی تحلیل نقش مشورتی دارد و کاربر می‌تواند بدون اجرای آن رکورد دارو را ثبت کند.

۲ـ۵ جهت متن و بومی‌سازی

برای فارسی‌سازی فقط عنوان‌ها را ترجمه نکردم. layout.tsx زبان صفحه را fa و جهت کلی را rtl قرار می‌دهد و DirectionProvider همین جهت را به اجزای رابط منتقل می‌کند. در CSS نیز فونت فارسی و فونت تک‌عرض لاتین جدا شده‌اند. مقدار آزمایش، دوز دارو و نام‌های لاتین در محل لازم dir=ltr دارند تا رشته‌ای مانند ۵۰۰ mg به‌هم نریزد و در عین حال جهت کل پرونده فارسی باقی بماند.

تاریخ ذخیره‌شده همان رشته استاندارد API باقی می‌ماند و Intl.DateTimeFormat فقط شکل نمایش آن را فارسی می‌کند. این جداسازی مرتب‌سازی و تبادل داده با سرور را ساده نگه می‌دارد. یک ایراد در تابع todayIsoDate وجود دارد: این تابع از toISOString استفاده می‌کند و روز UTC را برمی‌گرداند. نزدیک نیمه‌شب تهران ممکن است تاریخ ثبت‌شده یک روز با تاریخ محلی تفاوت داشته باشد. این حالت را در آزمون مرورگر اجرا نکردم و لازم است در ادامه با منطقه زمانی تهران آزمایش و اصلاح شود.

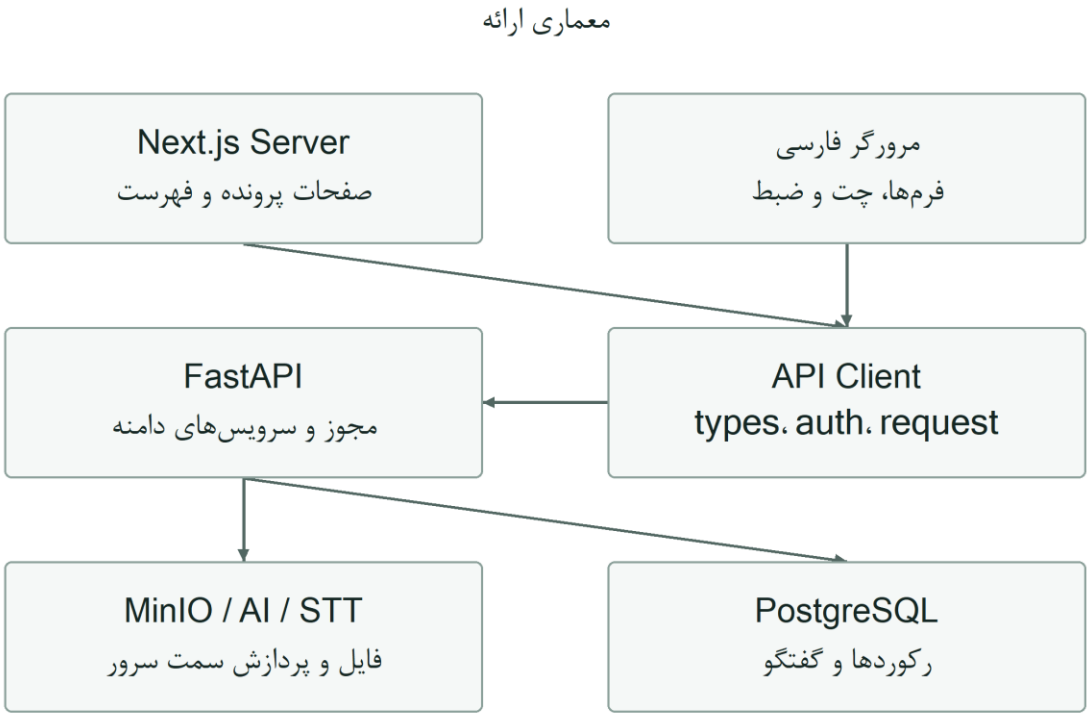
برای متن‌های ترکیبی، تنها راست‌چین‌کردن ظرف کافی نبود. مسیرهایی مانند /api/v۱/patients، نام فایل، مقدار ۵۰۰ mg و شناسه‌های MED-۱۲ اگر جهت مشخص نداشته باشند، کنار پرانتز و دونقطه جابه‌جا دیده می‌شوند. در اجزایی که مقدار فنی یک واحد مستقل است از dir=ltr و فونت تک‌عرض استفاده کردم و توضیح فارسی را در عنصر جدا نگه داشتم. همین قاعده در شماره تلفن، کد ملی، دوز، محدوده آزمایش و نام فایل نیز به کار رفت. در جدول‌ها ترتیب ستون‌ها را بر اساس خواندن فارسی چیدم، اما مقدار لاتین داخل سلول جهت خودش را حفظ می‌کند. این تصمیم علاوه بر ظاهر، روی کپی‌کردن مقدار هم اثر دارد؛ رشته‌ای که فقط با نشانه‌های نامرئی وارونه شده باشد ممکن است هنگام کپی به شکل دیگری درآید. در بازبینی خروجی، نمونه‌های فارسی کنار FastAPI و OpenRouter، مسیر API، نام متغیر محیطی و مقدار عددی را جداگانه دیدم. هنوز لازم است همین موارد با صفحه‌خوان و مرورگرهای دیگر هم آزمایش شوند، چون پشتیبانی الگوریتم دوجهته در همه محیط‌ها یکسان نیست.

فصل ۳ معماری و ساختار فرانت‌اند

۳ـ۱ ساختار مسیرها و مسئولیت پوشه‌ها

فرانت‌اند با App Router ساخته شده است. layout.tsx زبان، جهت و فونت را برای کل برنامه تنظیم می‌کند. صفحه اصلی برای معرفی محصول است، login ورود و ثبت‌نام را انجام می‌دهد، patients فهرست پرونده‌ها را نشان می‌دهد و patients/[id] فضای کار یک بیمار را باز می‌کند. در build نهایی، صفحه اصلی و ورود به‌صورت ایستا و دو مسیر بیمار به‌صورت پویا ساخته شدند. این بررسی ساخت برنامه را تأیید کرد؛ اتصال مسیرهای پویا به داده واقعی را جداگانه در محیط محلی آزمایش کردم.

اجزای پرونده در components/patients قرار دارند. پوشه copilot نمایش پاسخ و شواهد را نگه می‌دارد و summary کارت‌های نمای خلاصه را کنار هم می‌چینند. اجزای مشترک مانند حالت خطا، حالت خالی و پنجره تأیید در shared قرار گرفته‌اند و ui کنترل‌های عمومی را فراهم می‌کند. قرارداد داده، ارتباط، احراز هویت و قالب‌بندی نیز در lib هستند. با این تقسیم، Dialog فقط رفتار عمومی پنجره را پیاده می‌کند و DocumentRowActions متن و callback مخصوص حذف سند را به آن می‌دهد. شکل ۱ ارتباط این بخش‌ها را نشان می‌دهد.



شکل ۱ ـ معماری ارائه و مرزهای ارتباط در فرانت‌اند

۳ـ۲ فضای کار پرونده

پس از کنترل شناسه، صفحه پرونده زمینه بیمار و تایم‌لاین را هم‌زمان می‌خواند. در نمایش بزرگ، پرونده و ClinicalChat در دو ستون قرار می‌گیرند. در اندازه کوچک، کلاس order-first پنل چت را پیش از کارت‌های پرونده می‌آورد و در اندازه xl ترتیب دوباره عادی می‌شود. بنابراین نسخه موبایل صرفاً نمونه کوچک‌شده دسکتاپ نیست. در بازبینی موبایل دیدم پنل چت پیش از داده‌های خلاصه نمایش داده می‌شود؛ برای استفاده واقعی بهتر است اولویت این دو بخش با نظر کاربران بررسی شود.

در نمایش بزرگ، عرض ستون دستیار با درصد و سقف پیکسل محدود شده و ارتفاع آن به فضای دید وابسته است. فهرست پیام داخل پنل پیمایش می‌شود و ورودی در پایین ثابت می‌ماند؛ به این ترتیب، تاریخچه بلند ارتفاع کل صفحه را زیاد نمی‌کند. وجود پیمایش داخل پنل در کنار پیمایش صفحه می‌تواند برای صفحه‌کلید یا نمایشگر کوچک مشکل‌ساز شود. من چیدمان را در یک عرض موبایل بررسی کردم، اما آزمون روی دستگاه‌ها و اندازه‌های بیشتر انجام نشد.

۳ـ۳ هویت بیمار به‌عنوان زمینه ثابت

PatientIdentityBar نام، سن، جنسیت و شناسه‌های موجود را بالای فضای کار نشان می‌دهد و هشدار حساسیت شدید نیز در همین نوار قرار می‌گیرد. نوار sticky است تا هنگام پیمایش تب‌ها نام بیمار از دید خارج نشود. این تصمیم برای حفظ زمینه بیمار گرفته شد. در پروژه اندازه‌گیری نکردم که این چیدمان چند خطای انتخاب بیمار را کم می‌کند، اما در مرور طولانی پرونده دسترسی به مشخصات اصلی را حفظ می‌کند.

سن با calculateAge از تاریخ تولد محاسبه می‌شود و اگر تاریخ ثبت نشده باشد، سن نیز نمایش داده نمی‌شود. کد ملی و تلفن با dir=ltr و فونت mono از متن فارسی جدا شده‌اند تا ترتیب رقم‌ها حفظ شود. هشدار حساسیت علاوه بر رنگ، متن و آیکون دارد. شمارنده «مشکل فعال» فقط تعداد رکوردهای فعال را نشان می‌دهد و امتیاز خطر یا پیش‌بینی بالینی نیست.

۳ـ۴ تب‌ها و آدرس‌پذیری

WorkspaceTabs پنج گزینه summary، timeline، medications، labs و documents دارد. تب فعال از پارامتر tab خوانده می‌شود و parseWorkspaceTab مقدار ناشناخته را به summary برمی‌گرداند. هر گزینه یک Link واقعی است و aria-current وضعیت جاری را مشخص می‌کند. قراردادن تب در URL امکان پیوند مستقیم به یک بخش پرونده را فراهم می‌کند؛ برای نمونه، شاهد دارویی در چت می‌تواند کاربر را به medications ببرد و داده اصلی را در کنار پاسخ نشان دهد.

در HTML این گزینه‌ها Link هستند، نه یک ویجت tab با نقش‌های tablist و tabpanel. بنابراین رفتار صفحه‌کلید آن‌ها همان لینک است و با Tab و Enter کار می‌کند. هر تغییر تب نیز منطق صفحه سرور را

دوباره اجرا می‌کند و در نسخه فعلی زمینه و تایم‌لاین مستقل از تب انتخاب‌شده خوانده می‌شوند. این روش ساده است، ولی برای پرونده بزرگ می‌تواند درخواست‌های بیشتری از مقدار لازم ایجاد کند.

۳ـ۵ اجزای عمومی و قرارداد بازاستفاده

SectionCard و SectionHeader الگوی عنوان، شمارنده و کنش یک بخش را یکسان می‌کنند. CompactEmptyState نیز نبود رکورد را با متن کوتاه و گاهی راهنمای اقدام نمایش می‌دهد. ConfirmDialog وضعیت انتظار و خطا را در عملیات حساس نگه می‌دارد. مزیت این ساختار در یکپارچگی رفتار است: callback موفق پنجره را می‌بندد و callback ناموفق پیام را در همان محل نگه می‌دارد. حذف سند و حذف گفتگو می‌توانند این سازوکار را به اشتراک بگذارند، در حالی که توضیح پیامد حذف متفاوت باقی بماند.

کنترل‌های ui از اجزای عمومی پروژه ساخته شده‌اند و جهت RTL را از پوسته دریافت می‌کنند. رفتار پایه پنجره در Dialog قرار دارد، اما برچسب‌ها و ترتیب کنترل‌های هر فرم در جزء مصرف‌کننده تعریف می‌شود. این جداسازی نگه‌داری را ساده‌تر کرد، ولی تغییر یک جزء عمومی می‌تواند هم‌زمان روی فرم بیمار، سند، صوت و دارو اثر بگذارد. برای همین پس از چنین تغییرهایی باید همه این فرم‌ها دوباره بررسی شوند.

فصل ۴ قرارداد داده و ارتباط با API

۴ـ۱ یک مرز مشترک برای درخواست‌ها

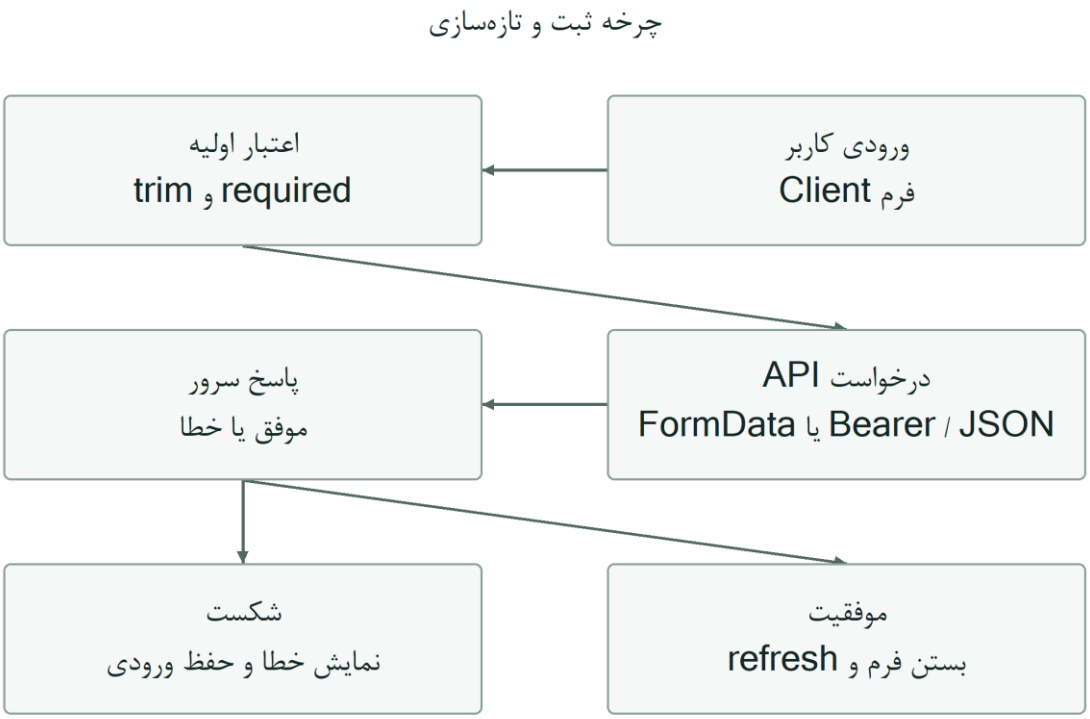
در فرانت‌اند، درخواست‌های مرتبط با هر بخش را در شیء api در فایل lib/api.ts متمرکز کردم. به این ترتیب، فرم‌ها نشانی endpoint و شیوه تفسیر پاسخ را جداگانه تکرار نمی‌کنند و فقط تابعی مثل createPatient یا uploadDocument را صدا می‌زنند. تابع عمومی request<T> سرآیندها، fetch، خطای شبکه، وضعیت HTTP و تبدیل پاسخ را مدیریت می‌کند. این انتخاب پیام قطع ارتباط و پاسخ بدون محتوا با وضعیت ۲۰۴ را در همه فرم‌ها یکسان نگه داشته است. یک ضعف آن این است که جزئیات خطای ۴۲۲ به پیام عمومی تبدیل می‌شود و کاربر همیشه نمی‌فهمد کدام فیلد ایراد دارد. چون این رفتار در یک تابع جمع شده، اصلاح آن هم از همان نقطه ممکن است.

مقدار API_BASE_URL از متغیر NEXT_PUBLIC_API_URL خوانده می‌شود و اگر تنظیم نشده باشد، نشانی محیط توسعه را می‌گیرد. پیشوند /api/v۱ در مقدار پیش‌فرض قرار دارد و توابع دامنه فقط مسیر نسبی، برای نمونه /patients، را به آن اضافه می‌کنند. این کار تکرار نسخه API در تک‌تک توابع را حذف می‌کند. هنگام اتصال صفحات سروری متوجه شدم نشانی مناسب مرورگر همیشه برای fetch سمت سرور مناسب نیست؛ چون مبدأ درخواست و روش رساندن توکن در این دو محیط فرق دارد. بنابراین در تنظیم این بخش، علاوه بر نام endpoint، محل اجرای درخواست و احراز هویت را هم در نظر گرفتم.

۴ـ۲ خواندن اولیه و ثبت تغییر

صفحه پرونده در بارگذاری اولیه، api.getPatientContext و api.getPatientTimeline را هم‌زمان اجرا می‌کند و نتیجه را در اختیار اجزای نمایش می‌گذارد. فرم‌ها عملیات ثبت یا ویرایش را در مرورگر انجام می‌دهند و پس از پاسخ موفق، router.refresh را فراخوانی می‌کنند. در نتیجه داده‌های سمت سرور دوباره خوانده می‌شوند و لازم نیست هر فرم نسخه محلی کل پرونده را خودش به‌روز کند. در این پروژه کتابخانه جداگانه‌ای برای cache به کار نرفته و بیشتر فرم‌ها نیز تغییر خوش‌بینانه روی کل پرونده انجام نمی‌دهند.

با این روش، سرور پس از ثبت همچنان مرجع داده است. برای مثال، فرم دارو بعد از پاسخ موفق آرایه داروها را به‌صورت دستی تغییر نمی‌دهد و صفحه فهرست تازه را از سرور می‌گیرد. مزیت این تصمیم آن است که قواعد مرتب‌سازی و فیلتر در چند فرم تکرار نمی‌شوند. در مقابل، بخش‌هایی از پرونده که تغییر نکرده‌اند نیز ممکن است دوباره خوانده شوند. در پرونده‌های بزرگ، همراه‌کردن متن کامل اسناد با هر بازآوری می‌تواند هزینه اضافی ایجاد کند. در این پروژه زمان و حجم این درخواست‌ها را اندازه‌گیری نکردم؛ برای نسخه بعدی، جداکردن داده نمایشی از متن حجیم و تازه‌سازی همان بخش تغییرکرده راه مناسب‌تری است.



شکل ۲ ـ چرخه ثبت تغییر و بازآوری داده در رابط

۴ـ۳ خطا به‌عنوان بخشی از قرارداد

در request، خطای fetch به ApiError با status برابر صفر تبدیل می‌شود. پاسخ ۴۲۲ متن عمومی اعتبار داده دارد و سایر وضعیت‌ها، در صورت وجود detail رشته‌ای، پیام سرور را حفظ می‌کنند. در خطای ۴۰۱ مرورگر، کلید localStorage حذف و کاربر به login منتقل می‌شود. در صفحات سرور، خطای ۴۰۱ با redirect مدیریت می‌شود. این تفاوت لازم است چون window در سرور وجود ندارد. برای خطای ۴۰۴ پرونده، notFound فراخوانی می‌شود، اما خطای ۴۰۳ به نمایش ErrorState می‌رسد و نباید به‌صورت «بیمار وجود ندارد» بازنویسی شود.

همه خطاهای HTTP به معنای انجام‌نشدن کامل عملیات نیستند. در مسیر چت، پیام کاربر پیش از فراخوانی مدل ذخیره می‌شود. اگر تولید پاسخ شکست بخورد، فرانت‌اند پیام موقت را حذف می‌کند و متن را به کادر ورودی برمی‌گرداند، در حالی که همان پرسش ممکن است در سرور باقی مانده باشد. در نتیجه، ارسال دوباره می‌تواند پیام تکراری بسازد. برای رفع این ابهام باید شناسه پیام در پاسخ خطا برگردد یا گفت‌وگو دوباره خوانده شود و وضعیت «پرسش ثبت شد، پاسخ تولید نشد» نمایش داده شود. این اصلاح هنوز در نسخه فعلی انجام نشده است.

۴ـ۴ جدول قراردادهای اصلی

در جدول ۱ توابع اصلی‌ای را آورده‌ام که رابط در گردش‌های اصلی استفاده می‌کند. نام تابع‌ها از api.ts و مسیرها نسبت به پیشوند API نوشته شده‌اند. جدول قرار نیست همه endpointهای بک‌اند را فهرست کند؛ هدف آن روشن‌کردن ارتباط فرم‌ها با سرویس است. نمونه payloadهای پیوست نیز ساختگی‌اند و فقط رابطه فیلد فرم و بدنه درخواست را نشان می‌دهند؛ هیچ داده واقعی بیمار در آن‌ها وجود ندارد.

جدول ۱ ـ قراردادهای اصلی مورد استفاده رابط

تابع یا کنش	مسیر و روش	قالب و نتیجه
listPatients	GET /patients	q، limit، offset؛ items و total
createPatient	POST /patients	JSON مشخصات؛ Patient
updatePatient	PATCH /patients/{id}	فیلدهای قابل تغییر؛ Patient
getPatientContext	GET /patients/{id}/context	داده بخش‌های پرونده
getPatientTimeline	GET /patients/{id}/timeline	items و total رویداد
uploadDocument	POST patients/{id}/documents/	FormData؛ رکورد سند
uploadVoiceNote	POST /patients/{id}/voice-notes	FormData؛ رکورد صوت
createNoteFromVoiceNote	POST /voice-notes/{id}/note	title و content؛ ثبت پس از تأیید
sendConversationMessage	POST conversations/{id}/messages/	content، medication_safety، provider/model اختیاری
aiMedicationSafetyCheck	POST patients/{id}/medications/ai-/safety-check	نام/دوز کاندید؛ تحلیل متن‌دار
agentChat	POST patients/{id}/agent/chat/	content؛ پاسخ بی‌حالت و tools_used

۴ـ۵ مدیریت وضعیت فرم در مرز درخواست

در بیشتر فرم‌ها سه وضعیت را جدا نگه داشتم: داده‌ای که کاربر وارد کرده، وضعیت درحال ارسال و پیام خطا. با آغاز درخواست، submitting فعال می‌شود تا ارسال دوباره اتفاق نیفتد، اما مقدار فیلدها پاک نمی‌شود. اگر پاسخ موفق باشد، پنجره بسته و داده صفحه از سرور تازه می‌شود؛ اگر درخواست شکست بخورد، پنجره باز می‌ماند و کاربر همان مقدارها را برای اصلاح در اختیار دارد. این الگو را در فرم بیمار، دارو، شرایط و حساسیت به کار بردم. تفاوت فرم چت این است که متن بلافاصله به شکل یک پیام موقت دیده می‌شود و ورودی خالی می‌گردد. در نتیجه، بازیابی شکست چت از فرم‌های عادی پیچیده‌تر است و باید هم پیام موقت و هم وضعیت ثبت در سرور را در نظر بگیرد.

برای reset فرم‌ها از effectهایی استفاده شده که هنگام بازشدن پنجره، state را از رکورد فعلی پر می‌کنند. این روش ویرایش یک رکورد را ساده کرده است، ولی ESLint در چند جزء آن را با قانون set-state-in-effect ناسازگار تشخیص داد. خاموش‌کردن قانون را راه‌حل مناسبی نمی‌دانم، چون مشکل اصلی امکان چند تغییر state و رندر اضافی است. راه بهتر این است که فرم با یک key وابسته به شناسه رکورد دوباره ساخته شود یا مقدار اولیه در یک reducer قرار گیرد و فقط با رویداد مشخص open/reset تغییر کند. در هر دو روش باید همان رفتار فعلی حفظ شود: بازکردن رکورد دیگر اطلاعات قبلی را نشان ندهد و خطای شبکه نیز داده‌ای را که کاربر تازه وارد کرده پاک نکند.

فصل ۵ ورود و مدیریت بیماران

۵ـ۱ مشاهده مستقیم ورود و ثبت‌نام

در شکل ۳ صفحه ورود اجراشده دیده می‌شود. فرم در حالت ورود فقط ایمیل و گذرواژه می‌گیرد و متن دکمه، مقصد بعدی را پرونده‌های بیماران معرفی می‌کند. جهت این دو ورودی را چپ‌به‌راست گذاشتم تا حروف لاتین، اعداد و نشانه‌های ایمیل در صفحه فارسی جابه‌جا نشوند. این تصویر برای بررسی چیدمان فرم گرفته شده است؛ موفقیت ورود را در سناریوی جداگانه آزمایش کردم.



شکل ۳ ـ کارت ورود پزشک در نمای دسکتاپ

با تغییر حالت همان صفحه، فیلد نام کامل هم ظاهر می‌شود. برای بررسی این مسیر، در محیط محلی یک حساب با نام و نشانی ساختگی ثبت کردم. برنامه بعد از ساخت حساب و ورود، فهرست بیماران را باز کرد. این

آزمایش فقط اتصال دو درخواست ثبت‌نام و ورود را در پیکربندی محلی نشان می‌دهد و جای بررسی هویت حرفه‌ای کاربران یا آزمون محیط تولید را نمی‌گیرد.

۵ـ۲ مرز اعتماد بین رابط و سرور

فرانت‌اند ورودی کاربر و وضعیت ورود را مدیریت می‌کند، اما اجازه دسترسی در سرور بررسی می‌شود. پنهان‌کردن یک دکمه یا حذف بیمار از فهرست به‌تنهایی از endpoint محافظت نمی‌کند. در بک‌اند، get_current_user توکن را اعتبارسنجی می‌کند و ensure_patient_access ارتباط کاربر با بیمار را می‌سنجد. در مسیرهای تودرتو از patient_from_id استفاده شده و برای منابعی مانند سند، دسترسی پس از یافتن همان منبع کنترل می‌شود. هنگام طراحی رابط، این کنترل‌های سرور را مرز اصلی اجازه در نظر گرفتم و نمایش یا پنهان‌سازی کنترل‌ها را فقط بخشی از تجربه کاربر دانستم.

طبق منطق موجود، مدیر به همه بیماران دسترسی دارد و کاربر با نقش doctor فقط پرونده‌های واگذارشده را می‌بیند. اگر همین کاربر بیمار تازه‌ای بسازد، ارتباط دسترسی او نیز ثبت می‌شود. این سیاست در routes/patients.py و dependencies.py پیاده شده است. نقش doctor در این نسخه یک نقش نرم‌افزاری است؛ ثبت‌نام MVP حسابی با همین نقش می‌سازد و بررسی پروانه یا تأیید سازمانی جداگانه‌ای در این مسیر وجود ندارد. در متن، واژه پزشک به کاربر هدف برنامه اشاره دارد و به‌خودی‌خود تأیید صلاحیت حرفه‌ای حساب نیست.

۵ـ۳ ورود و ثبت‌نام در رابط فعلی

در LoginPage، متغیر mode فرم را بین login و register جابه‌جا می‌کند. ایمیل و گذرواژه در هر دو حالت وجود دارند و نام کامل فقط در ثبت‌نام اضافه می‌شود. حداقل طول هشت نویسه برای گذرواژه هم در کنترل رابط و هم در schema سرور اعمال شده است. پیش از ارسال، ایمیل trim می‌شود. پس از ورود موفق، مسیر patients باز و صفحه تازه می‌شود؛ در مدت انتظار نیز دکمه اصلی غیرفعال است و خطا داخل فرم نمایش داده می‌شود. متن خطا را از نظر فنی بررسی کردم، اما برای سنجش میزان فهم آن توسط کاربران مطالعه جداگانه‌ای انجام نشد.

تابع register در lib/auth.ts ابتدا حساب را می‌سازد و بعد login را اجرا می‌کند؛ بنابراین این کار از دو درخواست مستقل تشکیل شده است. اگر درخواست اول موفق و ورود بعدی ناموفق شود، حساب ایجاد شده ولی کاربر هنوز وارد برنامه نشده است. پیام فعلی این حالت را با جزئیات کافی توضیح نمی‌دهد و ممکن است کاربر ثبت‌نام را دوباره انجام دهد. سرور اجازه خودثبت‌نام با نقش مدیر را نمی‌دهد و نقش پیش‌فرض doctor است. فرم نیز فیلدی برای درخواست نقش admin ندارد، هرچند کنترل قطعی این سیاست در سمت سرور انجام می‌شود.

۵ـ۴ توکن و نگهداری نشست

login، access_token را دریافت و setToken را صدا می‌زند. توکن در localStorage با کلید clinical_copilot_token و در cookie با نام access_token ذخیره می‌شود. cookie مسیر /، عمر هشت ساعت و SameSite=Lax دارد. چون با document.cookie نوشته می‌شود، HttpOnly نیست و در کد نوشتن آن Secure نیز تعیین نشده است. سرور Next این cookie را به Bearer تبدیل می‌کند تا صفحات سرور بتوانند API را بخوانند. این راهکار ساده دو نیاز مرورگر و SSR را پوشش می‌دهد، اما سطح دسترسی JavaScript به توکن را افزایش می‌دهد.

راهنمای OWASP نگهداری شناسه نشست حساس در localStorage را به‌دلیل دسترسی JavaScript توصیه نمی‌کند و برای cookie نیز ویژگی‌های محافظتی را بر اساس مدل تهدید مطرح می‌کند. با مقایسه این راهنما با کد پروژه، دسترسی JavaScript به توکن را یکی از محدودیت‌های معماری فعلی دانستم؛ در این بررسی آزمون نفوذ انجام نشد. راه بهتر می‌تواند مدیریت نشست در سرور Next و استفاده از cookie مناسب همراه یک proxy باشد. این تغییر باید با CSRF، مبدأ درخواست‌ها و نیاز SSR هماهنگ طراحی شود و فقط افزودن یک ویژگی cookie کافی نیست.

۵ـ۵ چرخه ورود به فهرست

فهرست بیماران را در دو حالت بررسی کردم: یک حساب بدون پرونده و یک حساب پس از ثبت بیمار ساختگی. در شکل ۴ پیام شروع کار و دکمه افزودن دیده می‌شود. شکل ۵ نیز ردیف «بیمار آزمایشی»، نشان یک داروی فعال و پیوند ورود به پرونده را نشان می‌دهد. هر دو تصویر از صفحه‌ای گرفته شده‌اند که داده را از API دریافت کرده است.



شکل ۴ ـ حالت خالی فهرست بیماران با مسیر آغاز ثبت



شکل ۵ ـ فهرست واقعی با یک بیمار کاملاً ساختگی

صفحه patients پس از دریافت searchParams، q را trim می‌کند و page را از رشته URL به عدد تبدیل می‌کند. مقدار صفحه با Math.max حداقل یک نگه داشته می‌شود و PAGE_SIZE برابر ۲۰ است. سپس listPatients با offset برابر حاصل ضرب شماره صفحه منهای یک در اندازه صفحه فراخوانی می‌شود. این رابطه، فهرست را به صفحه‌بندی سرور متصل می‌کند و فرانت‌اند همه بیماران را برای برش محلی دریافت نمی‌کند. نتیجه API شامل items و total است و تعداد صفحات از total محاسبه می‌شود. این ساختار در صورت افزایش تعداد پرونده‌ها از دریافت کامل فهرست جلوگیری می‌کند.

در تبدیل پارامتر page فقط حداقل یک کنترل شده و صحیح‌بودن عدد به‌طور صریح بررسی نمی‌شود. در نتیجه مقدار اعشاری یا بسیار بزرگ می‌تواند offset نامناسبی بسازد. این حالت را در مرورگر اجرا نکردم، اما از مسیر تبدیل مقدار روشن است که بهتر است برای شماره صفحه parser مشخصی تعریف شود و سرور نیز offset را اعتبارسنجی کند. در صفحه جزئیات بیمار این کنترل دقیق‌تر است و Number.isInteger همراه با شرط مثبت‌بودن شناسه استفاده می‌شود.

۵ـ۶ جست‌وجو و حفظ زمینه پیمایش

فرم جست‌وجو action برابر /patients دارد و نام کنترل q است. ارسال فرم URL قابل بازگشت می‌سازد و مقدار defaultValue از query جاری پر می‌شود. کاربر می‌تواند عبارت را در نوار جست‌وجو ببیند و با پیوند پاک‌کردن، به فهرست بدون q برگردد. عنوان شمارنده تعداد بیمار نیز از total پاسخ می‌آید، نه تعداد سطرهای صفحه. این تفاوت برای صفحه دوم و بعدی مهم است؛ شمارنده کل فهرست را نشان می‌دهد و pagination محل فعلی را تعیین می‌کند. رفتار فهرست با URL قابل پیگیری‌تر از state پنهان یک کنترل جست‌وجو است.

رابط برای نبود پرونده و نبود نتیجه جست‌وجو حالت جدا دارد. در حالت نخست، دکمه افزودن بیمار مسیر شروع را فراهم می‌کند. در حالت دوم، متن به جست‌وجوی فعلی اشاره و پاک‌کردن آن را پیشنهاد می‌کند. اگر API خطا دهد، ErrorState نشان داده می‌شود و وضعیت خالی جای خطا نمی‌نشیند. این تمایز مهم است، زیرا نمایش «بیماری وجود ندارد» هنگام قطع سرویس می‌تواند کاربر را به ایجاد پرونده تکراری سوق دهد. حفظ معنای نبود داده، نبود نتیجه و نبود ارتباط از الزامات اصلی این صفحه است و در شاخه‌های شرطی کد قابل مشاهده است.

۵ـ۷ فرم ایجاد و ویرایش بیمار

شکل ۶ پنجره ایجاد بیمار را پیش از ورود داده نشان می‌دهد. نام و نام خانوادگی الزامی‌اند و کد ملی، تاریخ تولد، جنسیت، اطلاعات تماس و نشانی در همان فرم قرار دارند. برای آزمون، فقط اطلاعات ساختگی وارد کردم و بعد از ثبت، ردیف بیمار در فهرست ظاهر شد. صحت هویت یا همه حالت‌های اعتبارسنجی در این سناریو بررسی نشد.



شکل ۶ ـ پنجره‌ی واقعی ایجاد بیمار و فیلدهای هویتی

PatientFormDialog بر حسب وجود patient در props، حالت ایجاد یا ویرایش را انتخاب می‌کند. هنگام بازشدن، مقادیر از رکورد یا رشته خالی پر می‌شوند و error پاک می‌شود. کنترل‌های نام و نام خانوادگی required و دارای سقف طول‌اند. کد ملی inputMode عددی و pattern ده‌رقمی دارد، اما required نیست و در خالی‌بودن null ارسال می‌شود. جنسیت با مجموعه گزینه نوع‌دار عرضه می‌شود و تاریخ به‌صورت date با جهت لاتین است. این کنترل‌ها اعتبار اولیه را تسهیل می‌کنند؛ منطق سرور برای کد تکراری، تاریخ نامعتبر یا حد طول مرجع نهایی است.

در handleSubmit، یک payload از state جاری ساخته و یکی از createPatient یا updatePatient فراخوانی می‌شود. submitting قبل از درخواست فعال است و در finally خاموش می‌شود. در موفقیت، پنجره بسته و صفحه refresh می‌شود؛ در خطا، پنجره باز می‌ماند و مقدارها حفظ می‌شوند. این ترتیب از شروع مجدد فرم پس از شکست جلوگیری می‌کند. بااین‌حال، پیام عمومی ۴۲۲ در api.ts اطلاعات فیلد خاص

را حذف می‌کند و برای کاربر مشخص نمی‌شود کدام مقدار باید اصلاح شود. بهبود آینده می‌تواند خطاهای سرور را به کنترل مربوط متصل کند و خلاصه خطا را برای فناوری کمکی قابل اعلان سازد.

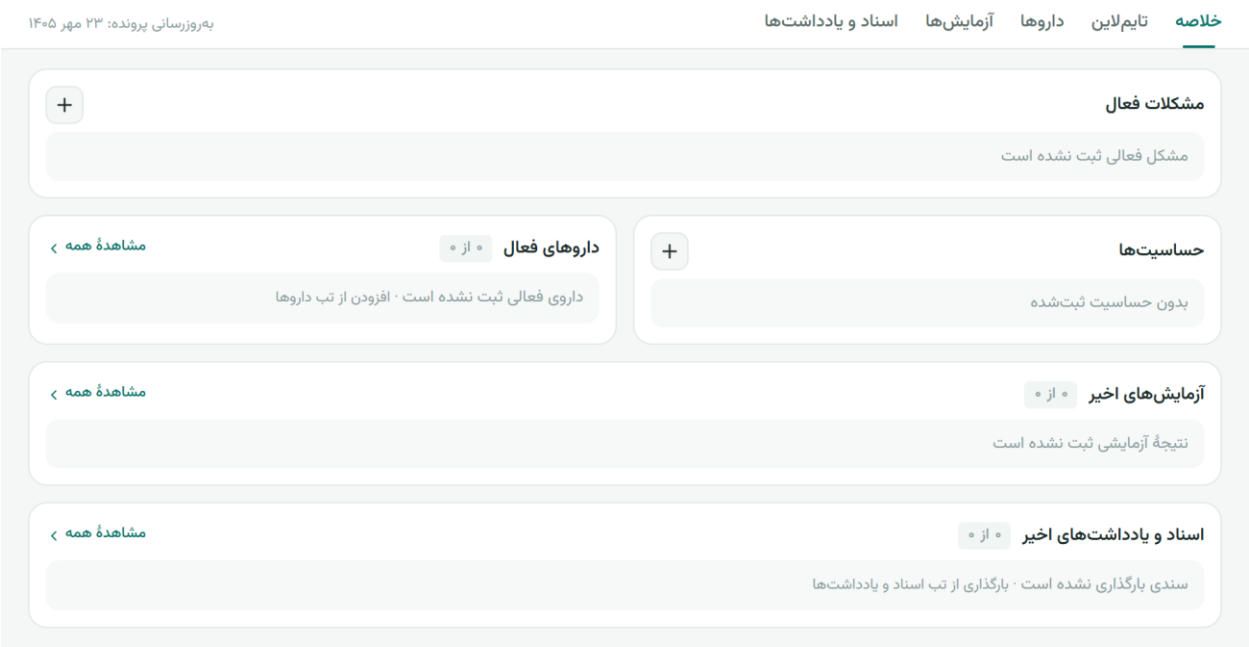
در حالت ویرایش، اطلاعات اولیه از همان patientای می‌آید که به پنجره داده شده است. عنوان پنجره و متن دکمه نیز از «بیمار جدید» به «ویرایش بیمار» تغییر می‌کنند تا کاربر بداند عملیات فعلی ساخت رکورد نیست. با این حال، نام بیمار فقط داخل فیلدها دیده می‌شود و اگر کاربر چند پرونده را در تب‌های جدا باز کرده باشد، یک نشانه هویتی ثابت داخل پنجره می‌تواند احتمال ویرایش پرونده اشتباه را کمتر کند. برای نسخه بعدی، نمایش نام کامل و شناسه پرونده در سرآیند پنجره و مقایسه دوباره patient.id پیش از ارسال پیشنهاد می‌شود. کنترل اصلی دسترسی همچنان در سرور باقی می‌ماند، ولی رابط باید زمینه عملیات را نیز روشن نگه دارد.

پس از ثبت موفق، router.refresh داده را دوباره از سرور می‌گیرد. این تصمیم باعث می‌شود مقدارهای نهایی، از جمله تغییرهایی که سرور هنگام ذخیره اعمال کرده، در صفحه دیده شوند. اگر درخواست به علت کد ملی تکراری یا مقدار نامعتبر رد شود، پنجره بسته نمی‌شود و کاربر می‌تواند همان فیلد را اصلاح کند. در وضعیت فعلی فقط پیام کلی بالای فرم دیده می‌شود؛ اتصال خطای سرور به فیلد مربوط، قرار دادن aria-describedby روی کنترل و انتقال تمرکز به نخستین خطا، مسیر اصلاح را کوتاه‌تر می‌کند. این تغییرها باید طوری انجام شوند که متن واردشده در سایر فیلدها حفظ شود و کاربر مجبور به پرکردن دوباره کل فرم نباشد.

فصل ۶ پرونده بیمار و مدیریت اسناد

۶ـ۱ خلاصه به‌عنوان نمای منتخب داده

شکل ۷ نمای خلاصه پرونده ساختگی را پس از ورود نشان می‌دهد. عبارت‌های «مشکل فعالی ثبت نشده» و «نتیجه آزمایشی ثبت نشده» فقط خالی‌بودن رکوردهای همین نمونه را بیان می‌کنند. در زمان گرفتن این تصویر، شمارنده دارو صفر بود. بعد از ثبت یک داروی نمایشی، شمارنده در نمای موبایل به یک تغییر کرد و پیوند «مشاهده همه» همچنان در دسترس بود. با این آزمایش، به‌روزرسانی شمارنده از داده پرونده را بررسی کردم.



شکل ۷ ـ کارت‌های خلاصه‌ی پرونده در فضای کار واقعی

SummaryView شرایط، حساسیت‌ها، داروها، آزمایش‌ها و اسناد را از داده پرونده دریافت می‌کند و در این جزء فراخوانی مدل زبانی انجام نمی‌شود. منظور از «خلاصه» در این صفحه، انتخاب و گروه‌بندی بخش‌های مهم پرونده است. تعداد داروهای فعال و آزمایش‌های اخیر مستقیماً از آرایه‌های پاسخ به دست می‌آیند. کار من در این بخش انتخاب اطلاعات، ترتیب دیداری و فراهم‌کردن مسیر ورود به جزئیات بود؛ query و مدل داده سرور در بخش مشترک پروژه و حوزه اصلی عرفان قرار داشت.

در نمای خلاصه، هشدار حساسیت پیش از مشکلات فعال، داروها، آزمایش‌ها و اسناد اخیر قرار گرفته است. این ترتیب را برای دیده‌شدن زودتر اطلاعات حساس انتخاب کردم، اما در پروژه مقایسه کاربری میان چند چیدمان انجام نشد. در نتیجه نمی‌توان از روی این ترتیب درباره کاهش خطای پزشکی نتیجه‌گیری کرد. برای ارزیابی دقیق‌تر باید سناریوهای واقعی مرور پرونده تعریف شود و زمان و خطای یافتن اطلاعات با کاربران واجد صلاحیت سنجیده شود. آزمون فعلی فقط با داده‌های ساختگی انجام شد.

۶ـ۲ اعلان حساسیت و رابطه با ثبت داده

CriticalAlerts آرایه حساسیت را به severe و سایر موارد تقسیم می‌کند. اگر حساسیت شدید وجود نداشته باشد اما موارد دیگر ثبت شده باشند، یک اعلان آرام با فهرست آن‌ها نشان داده می‌شود. اگر severe وجود داشته باشد، نخستین حساسیت در عنوان برجسته می‌شود و دیگر حساسیت‌ها و واکنش موجود در زیرخط قرار می‌گیرند. اگر هیچ رکوردی نباشد، این جزء چیزی نمایش نمی‌دهد. نبود اعلان نباید از نظر بالینی به «بیمار قطعاً حساسیت ندارد» تبدیل شود، زیرا مدل داده فقط نبود رکورد در پاسخ را نشان می‌دهد.

اگر چند حساسیت یا واکنش طولانی وجود داشته باشد، متن زیر هشدار با truncate کوتاه می‌شود و فهرست کامل‌تر در AllergyList دیده می‌شود. با این حال، باید جداگانه بررسی شود که همه جزئیات واکنش برای فناوری‌های کمکی نیز قابل دسترسی باشند. بک‌اند حساسیت شدید را جلوتر می‌آورد، ولی ترتیب رکوردهای هم‌سطح را نمی‌توان رتبه‌بندی بالینی خطر دانست. در طراحی فعلی، جایگاه بصری فقط برای برجسته‌کردن رکورد شدید استفاده شده است.

۶ـ۳ داروها و شرایط فعال

ActiveProblems شرایط active را فیلتر و دکمه افزودن و کنش سطر را ارائه می‌کند. این نما تمرکز بر وضعیت جاری دارد، ولی شرایط برطرف‌شده در آن دیده نمی‌شوند. ActiveMedications نیز فقط active را نشان می‌دهد و با «مشاهده همه» به تب داروها می‌رود. در شمارنده این بخش، تعداد فعال از کل رکوردهای دارویی ارائه می‌شود. این الگو به کاربر می‌گوید نمای خلاصه برشی از مجموعه است و برای تاریخچه باید مسیر کامل را باز کند. برای شرایط، مسیر مشابه «مشاهده همه» در جزء بررسی‌شده دیده نمی‌شود و عدم تقارن در پوشش تاریخچه باید ثبت شود.

نام عمومی دارو و مقدار دوز را با جهت LTR و فونت تک‌عرض نمایش دادم تا ترکیب اعداد و حروف لاتین به‌هم نریزد. واحد و مقدار از داده ذخیره‌شده خوانده می‌شوند و جزء خلاصه محاسبه تازه‌ای روی آن‌ها انجام نمی‌دهد. ورودی دوز در نسخه فعلی آزاد است و به واژگان استانداردی مانند RxNorm یا UCUM متصل نشده؛ بنابراین اگر کاربر قالب متفاوتی وارد کند، همان مقدار در خلاصه نمایش داده می‌شود. در ادامه، استانداردسازی داده باید هم‌زمان با بازطراحی کنترل ورود انجام شود، چون تغییر ظاهر عدد به‌تنهایی معنای آن را معتبر نمی‌کند.

۶ـ۴ آزمایش‌های اخیر و محدودیت برداشت

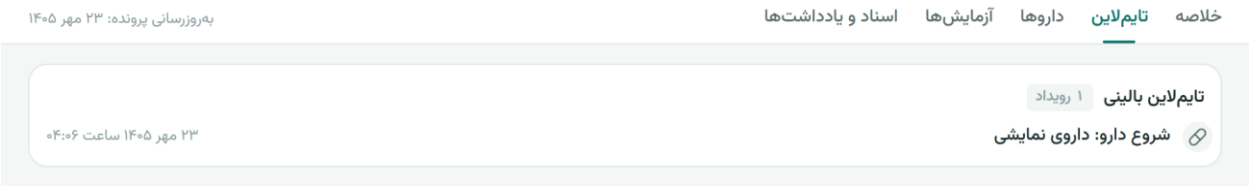
RecentLabs پنج رکورد ابتدای آرایه را نمایش می‌دهد و پیوند «مشاهده همه» کاربر را به تب labs می‌برد. ترتیب اخیر از سرور می‌آید و این جزء دوباره بر اساس تاریخ مرتب نمی‌کند. value، unit و reference_range رشته‌های قراردادند و flag نیز از پاسخ خوانده می‌شود. رابط فقط بر پایه مقدار

normal، abnormal یا critical کلاس رنگ را انتخاب می‌کند و خودش عدد را با محدوده مرجع مقایسه نمی‌کند. بنابراین تشخیص غیرطبیعی‌بودن آزمایش در فرانت‌اند انجام نمی‌شود.

اگر محدوده مرجع موجود نباشد، خط تیره نشان داده می‌شود و در نبود flag نیز نشان وضعیت نمایش داده نمی‌شود. این رفتار «نامعلوم» را از «طبیعی» جدا نگه می‌دارد. مقدار و محدوده را LTR قرار دادم، چون برچسب آزمایش و واحد ممکن است فارسی و لاتین را با هم داشته باشند. متن حالت خالی LabsView به استخراج از اسناد اشاره می‌کند، اما سرویس فعلی استخراج ساختاریافته همه نتایج آزمایش را تضمین نمی‌کند. این متن رابط باید در نسخه بعدی دقیق‌تر با توان واقعی پردازش سند هماهنگ شود.

۶ـ۵ ساخت و نمایش روند زمانی

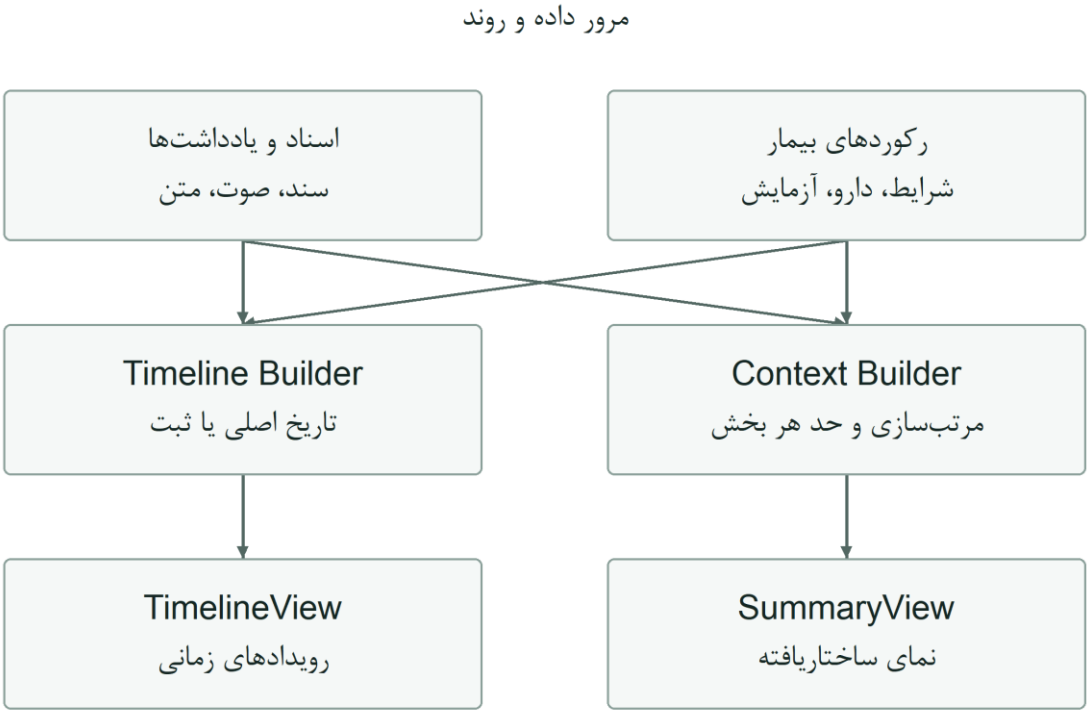
برای بررسی پیوستگی فرم دارو و خط زمانی، رکوردی با نام «داروی نمایشی» از فرم عادی ثبت کردم. در شکل ۸ رویداد «شروع دارو» در تب تایم‌لاین دیده می‌شود. این سناریو فقط مسیر ثبت یک دارو تا نمایش رویداد را در محیط محلی پوشش داد؛ همه منبع‌های خط زمانی و دقت تاریخ‌های بالینی در آن آزمایش نشدند.



شکل ۸ ـ رویداد تایم‌لاین پس از ثبت داروی نمایشی

build_timeline در سرور از رکوردهای واقعی شرایط، داروها، آزمایش‌ها، اسناد، صوت و یادداشت رویداد می‌سازد. این مسیر فقط جدول clinical_events را نمایش نمی‌دهد. تاریخ اصلی در صورت وجود به datetime تبدیل و در نبود آن created_at جایگزین می‌شود. خروجی به ترتیب جدیدتر ارائه می‌شود و هر رویداد source_type و source_id دارد. در رابط، TimelineView یک ol با خط جانبی و آیکون نوع رویداد می‌سازد. نقش فرانت‌اند ارائه عنوان، زمان، توضیح و تمایز بصری رویداد است و تولید ترتیب و داده در سرور انجام می‌شود.

اگر تاریخ اصلی یک رویداد موجود نباشد، سرور created_at را جایگزین می‌کند تا رویداد از خط زمانی حذف نشود. این زمان لزوماً با زمان رخداد بالینی برابر نیست؛ برای مثال ممکن است یک سند قدیمی امروز بارگذاری شود. در TimelineItem فعلی فیلدی برای توضیح منشأ زمان وجود ندارد و رابط نمی‌تواند «زمان رخداد» را از «زمان ثبت» جدا نشان دهد. افزودن time_source به پاسخ و نمایش برچسب مناسب، این ابهام را برطرف می‌کند. این فیلد در نسخه فعلی وجود ندارد.



شکل ۹ ـ منشأ داده‌های نمای خلاصه و تایم‌لاین

۶ـ۶ نقش سند در پرونده

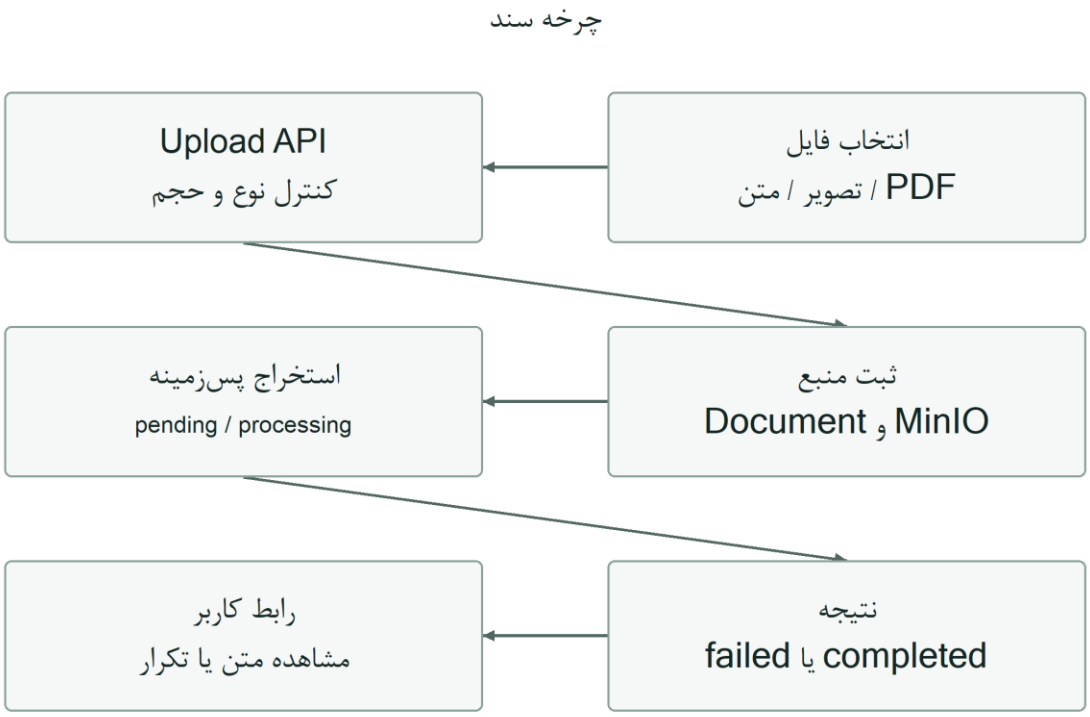
در شکل ۱۰ سه ناحیه اسناد، یادداشت صوتی و یادداشت بالینی در پرونده خالی دیده می‌شوند. برای هر ناحیه متن حالت خالی و دکمه شروع کنش جداگانه در نظر گرفتم. هنگام گرفتن تصویر، ذخیره‌سازی شیء در محیط نمایشی آماده نبود؛ به همین دلیل فایل پزشکی یا نمونه ساختگی به سرور ارسال نکردم.



شکل ۱۰ ـ تب اسناد و یادداشت‌ها با سه نوع رکورد

برای نمایش سند، فایل اصلی را از فراداده و متن استخراج‌شده جدا نگه داشتم. فایل می‌تواند PDF، تصویر یا متن ساده باشد؛ فراداده عنوان، نوع، نام، اندازه و وضعیت را نگه می‌دارد و extracted_text نتیجه پردازش است. نوع Document فیلدهای ai_summary و analysis_status را نیز دارد که خروجی مشتق‌شده را مشخص می‌کنند. جدابودن این لایه‌ها به کاربر اجازه می‌دهد نتیجه پردازش را با منبع اصلی مقایسه کند. patient_id نیز سند را به پرونده متصل می‌کند و دسترسی باید پیش از ارائه فایل کنترل شود.

سمت سرور فایل را در ذخیره‌سازی شیء نگه می‌دارد و فراداده سند را در پایگاه داده ثبت می‌کند. parser و OCR در حوزه اصلی عرفان پیاده شد، اما برای طراحی رابط لازم بود وضعیت خروجی آن‌ها را دقیق بشناسم. پاسخ موفق آپلود فقط دریافت فایل را تأیید می‌کند و لزوماً به معنای موفقیت OCR نیست. به همین دلیل extraction_status را در فهرست سند نمایش دادم. در شکل ۱۱ مرحله قابل مشاهده برای کاربر را از پردازش پس‌زمینه جدا کرده‌ام. استخراج ساختاریافته نتیجه‌های آزمایش در این گردش نمایش داده نمی‌شود.



شکل ۱۱ ـ چرخه بارگذاری سند و نمایش وضعیت استخراج

۶ـ۷ انتخاب فایل و اطلاعات کمکی

شکل ۱۲ پنجره بارگذاری را با انتخاب فایل، عنوان و نوع سند نشان می‌دهد. در این مرحله بازشدن پنجره، ترتیب کنترل‌ها و انتخاب نوع سند را بررسی کردم. به‌دلیل آماده‌نبودن ذخیره‌سازی شیء در محیط نمایشی، آپلود، OCR و تحلیل سند در این سناریو اجرا نشدند.



شکل ۱۲ ـ پنجره‌ی انتخاب فایل و فراداده‌ی سند

AddDocumentButton پنجره بارگذاری را باز می‌کند. stateهای title، file و docType ورودی را نگه می‌دارند و useRef برای پاک‌کردن کنترل فایل هنگام بازشدن استفاده می‌شود. accept پسوندهای PDF، WebP، JPEG، PNG و TXT را پیشنهاد می‌دهد. عنوان اختیاری است و در خالی‌بودن به‌صورت undefined کنار گذاشته می‌شود تا سرور نام فایل را به‌عنوان پیش‌فرض به کار گیرد. نوع سند از documentTypeOptions انتخاب می‌شود و مقدار اولیه other است. تعیین نوع توسط کاربر یک برچسب طبقه‌بندی است؛ تحلیل خودکار محتوا یا تأیید بالینی نوع سند از آن نتیجه نمی‌شود.

نام فایل و اندازه انتخاب‌شده پیش از ارسال نمایش داده می‌شوند. در این نما، جهت LTR برای رشته نام فایل به کار رفته و اندازه بر حسب KB محاسبه می‌شود. کنترل accept مانع قطعی ارسال فایل نامجاز نیست و باید با MIME و محدودیت اندازه سرور همراه شود. نسخه فعلی رابط محدودیت ۲۰ مگابایت را در توضیح بیان می‌کند، اما بررسی صریح اندازه پیش از درخواست در handleSubmit دیده نمی‌شود. بنابراین یک فایل بزرگ ممکن است ارسال شود و بعد خطای سرور بگیرد. بررسی محلی آینده می‌تواند این رفت‌وبرگشت را کاهش دهد، بدون آن‌که جای کنترل سرور را بگیرد.

۶ـ۸ وضعیت استخراج و زمان اطلاع کاربر

ExtractionStatus چهار مقدار pending، processing، completed و failed دارد. نگاشت فارسی هر مقدار در format.ts تعریف شده و کلاس نشان در صفحه پرونده، موفقیت، شکست و حالت درحال پردازش را از هم جدا می‌کند. آیکون و عنوان سند نیز امکان مرور فهرست را فراهم می‌کنند. completed یعنی پردازش متن در این قرارداد پایان یافته، نه این‌که محتوای پزشکی آن به‌طور کامل صحیح استخراج شده است. OCR ممکن است خطا داشته باشد و PDF ممکن است متن ناقص بدهد؛ رابط به همین دلیل فایل اصلی را معتبرترین منبع معرفی می‌کند.

پس از آپلود یا درخواست پردازش مجدد، صفحه یک بار refresh می‌شود، اما فهرست وضعیت را به‌طور پیوسته پایش نمی‌کند. بنابراین ممکن است تغییر processing به completed تا بازآوری بعدی در رابط دیده نشود. نشان processing بدون زمان یا راه بررسی دوباره، برای کاربر شبیه توقف عملیات به نظر می‌رسد. برای نسخه بعدی، polling محدود فقط برای رکوردهای در حال پردازش و یک دکمه بازخوانی دستی مناسب است. جدول ۲ رفتار فعلی و این نیاز را از هم جدا نشان می‌دهد. در آزمون حاضر OCR واقعی اجرا نشد.

جدول ۲ ـ وضعیت استخراج و رفتار قابل مشاهده رابط

وضعیت	رفتار جاری	برداشت مجاز
pending	نمایش نشان در انتظار	پردازش هنوز نتیجه نهایی ندارد
processing	نشان درحال پردازش	اتمام باید در بازخوانی بعدی دیده شود
completed	نمایش متن در صورت غیرخالی بودن	پایان استخراج؛ صحت پزشکی تأیید نشده
failed	نشان شکست و کنترل تکرار	خطای پردازش، نه لزوماً فایل ناموجود

فصل ۷ گفت‌وگوی بالینی و نمایش شواهد

۷ـ۱ نقش پنل دستیار در فضای پرونده

شکل ۱۳ حالت آغازین پنل را در پرونده ساختگی نشان می‌دهد. عنوان پرسش، توضیح وابستگی پاسخ به اطلاعات ثبت‌شده و چهار پیشنهاد آماده در یک ناحیه قرار دارند. نام بیمار در سربرگ نیز با پرونده بازشده یکسان است. هنگام گرفتن این تصویر پرسشی به مدل نفرستادم؛ بنابراین تصویر برای بررسی چیدمان آغاز گفت‌وگو است و کیفیت پاسخ یا منبع‌دهی مدل را ارزیابی نمی‌کند.

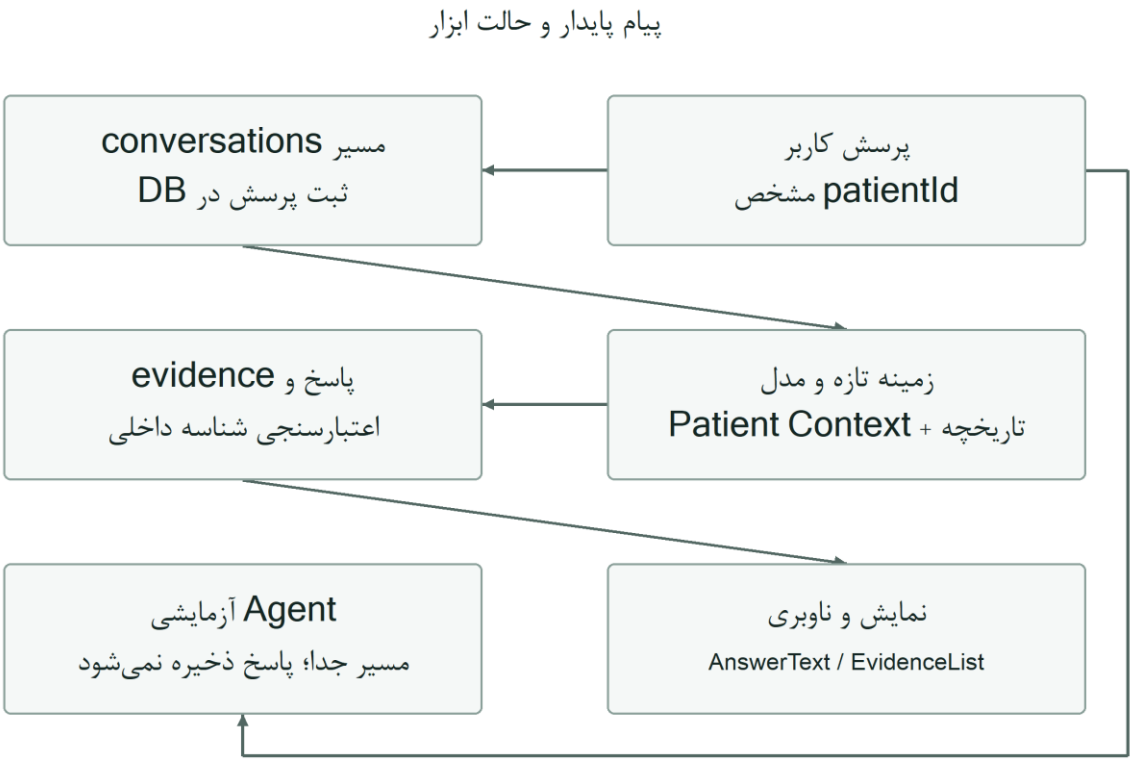


شکل ۱۳ ـ حالت خالی پنل گفت‌وگو و پرسش‌های پیشنهادی

ClinicalChat یک جزء Client است که patientId و نام بیمار را دریافت می‌کند. نام بیمار در سربرگ پنل، پاسخ‌ها را به زمینه انتخاب‌شده متصل می‌کند. چهار پرسش پیشنهادی در حالت خالی ارائه می‌شوند: مرور وضعیت، داروها، آزمایش‌های غیرطبیعی و تغییرات اخیر. این پیشنهادها شروع تعامل را ساده می‌کنند، ولی خودشان اجرای تحلیل یا صحت پاسخ را ثابت نمی‌کنند. متن ثابت پنل، سامانه را کمک تصمیم‌گیری معرفی می‌کند و مسئولیت نهایی را به پزشک می‌سپارد. چت در کنار پرونده قرار دارد تا کاربر بتواند تفسیر را با داده مقایسه کند، نه آن‌که پاسخ مدل جای داده را بگیرد.

در کد سه مسیر جدا وجود دارد: گفت‌وگوی پایدار با پیام‌های ذخیره‌شده، endpoint قدیمی و بی‌حالت chat/، و مسیر آزمایشی Agent. پنل اصلی اکنون از conversations استفاده می‌کند و فقط با فعال‌شدن

حالت ابزار به agentChat می‌رود. تابع askClinicalQuestion هنوز در api.ts وجود دارد، ولی مسیر عادی پنل از آن استفاده نمی‌کند. در شکل ۱۴ مسیر ذخیره پرسش و دریافت پاسخ را از Agent جدا کرده‌ام تا رفتار نسخه فعلی روشن باشد.



شکل ۱۴ ـ گردش پیام پایدار و مسیر جداگانه آزمایشی Agent

۷ـ۲ ایجاد، انتخاب و حذف گفتگو

handleNewConversation یک گفتگوی خالی در سرور ایجاد می‌کند، آن را به ابتدای فهرست محلی می‌افزاید، activeId را تنظیم و ورودی و پیام‌ها را پاک می‌کند. در ارسال نخستین پرسش نیز اگر activeId موجود نباشد، همین ایجاد به‌صورت ضمنی انجام می‌شود. عنوان اولیه در بک‌اند مشخص است و پس از نخستین پیام، از متن آن به‌صورت قطعی مشتق می‌شود؛ مدل برای تولید عنوان استفاده نمی‌شود. در رابط، پس از تبادل موفق، فهرست دوباره خوانده می‌شود تا عنوان و ترتیب فعالیت هماهنگ شوند. شکست این تازه‌سازی ثانویه، پاسخ موجود را حذف نمی‌کند.

انتخاب گفتگو ابتدا activeId و turns را تغییر می‌دهد و سپس جزئیات را می‌خواند. حذف با ConfirmDialog انجام می‌شود و پس از موفقیت، گفتگو از فهرست محلی حذف می‌شود؛ اگر مورد دیگری بماند، همان انتخاب می‌شود و در غیر این صورت پنل خالی می‌ماند. این منطق، یک مسیر ادامه مشخص دارد. بااین‌حال، رقابت چند انتخاب سریع و عملیات هم‌زمان نیاز به کنترل نسخه درخواست دارد، همان‌طور که در

فصل مدیریت وضعیت توضیح داده شد. دکمه حذف نیز باید معنای از دست‌دادن همه پیام‌های گفتگو را به کاربر اعلام کند و متن فعلی این پیامد را مطرح می‌کند.

۷ـ۳ ورودی، ارسال و حفظ متن

ورودی پرسش حداکثر ۲۰۰۰ نویسه دارد و مقدار خالی پس از trim ارسال نمی‌شود. Enter برای ارسال و Shift+Enter برای خط جدید استفاده شده و isComposing مانع ارسال ناخواسته هنگام ترکیب ورودی می‌شود. دکمه ارسال در حالت انتظار یا نبود متن غیرفعال است. handleSubmit متن را از state می‌گیرد، ورودی را پاک و send را اجرا می‌کند. send یک پیام کاربر محلی به فهرست می‌افزاید و loading را فعال می‌کند. این نمایش اولیه، صرفاً بازتاب قصد ارسال است و هنوز سند مستقل موفقیت ثبت در سرور نیست.

در پاسخ موفق، assistant_message همراه محتوا، شواهد و زمان سرور به فهرست اضافه می‌شود. قرارداد user_message را هم برمی‌گرداند، اما پیام اولیه کاربر با زمان محلی ساخته می‌شود و همان لحظه با نسخه سرور جایگزین نمی‌شود؛ بنابراین زمان ظاهری پرسش ممکن است تا بازخوانی بعدی کمی فرق داشته باشد. در catch، پیام موقت حذف و متن پرسش به ورودی برمی‌گردد. این کار از گم‌شدن متن جلوگیری می‌کند، ولی اگر سرور پرسش را پیش از شکست مدل ذخیره کرده باشد، رابط با تاریخچه پایدار هماهنگ نمی‌ماند. این حالت باید در طراحی بازیابی اصلاح شود.

۷ـ۴ تاریخچه گفتگو و زمینه بیمار

سرور در send_conversation_message ابتدا تاریخچه پیش از پیام جاری را می‌خواند و حداکثر ۲۰ پیام اخیر را در ترتیب زمانی برای مدل آماده می‌کند. سپس پیام کاربر ثبت می‌شود و زمینه بیمار به‌صورت تازه از داده فعلی ساخته می‌شود. تاریخچه گفتگو و زمینه بیمار دو مفهوم متفاوت‌اند: اولی جریان پرسش و پاسخ را نگه می‌دارد و دومی وضعیت فعلی پرونده را برای هر درخواست می‌سازد. ذخیره متن پاسخ به‌تنهایی ثابت نمی‌کند تمام داده مورد استفاده برای پاسخ قدیمی هنوز همان است. در مدل فعلی، snapshot کامل زمینه همراه پیام ذخیره نمی‌شود.

ساخت دوباره زمینه بیمار برای هر درخواست باعث می‌شود پاسخ تازه از داده فعلی استفاده کند، اما بازسازی شرایط تاریخی را دشوار می‌سازد. اگر دارویی بعد از پاسخ تغییر کند، کاربر متن قدیمی را کنار پرونده جدید می‌بیند. زمان پیام سرنخ می‌دهد، ولی نسخه داده یا snapshot زمینه همراه پیام ذخیره نشده است. برای ادامه پروژه می‌توان نسخه رکوردها یا خلاصه شواهد زمان پاسخ را نگه داشت؛ این تصمیم باید با هزینه ذخیره‌سازی و حریم خصوصی سنجیده شود.

۷ـ۵ شواهد داخلی و مراجعه به اصل

بک‌اند برای رکوردهای زمینه شناسه‌هایی با پیشوند NOTE، DOC، LAB، MED و VOICE می‌سازد. مدل باید فقط یکی از همین شناسه‌ها را برگرداند و validate_evidence هر مورد را با نمایه همان بیمار تطبیق می‌دهد؛ شناسه نامعتبر، ساختگی یا متعلق به بیمار دیگر حذف می‌شود. این کنترل مشخص می‌کند منبع واقعاً در زمینه بیمار وجود داشته است، اما درستی معنایی همه جمله‌های پاسخ را به‌تنهایی تضمین نمی‌کند. «شواهد بالینی» در این بخش به رکوردهای داخل پرونده اشاره دارد و بازیابی مقاله پزشکی بیرونی در این مسیر انجام نمی‌شود.

EvidenceList شناسه فنی را از نمایش اصلی کنار می‌گذارد و نوع منبع، توضیح و مسیر مراجعه را نشان می‌دهد. اگر document_id موجود باشد، لینک فایل ساخته می‌شود؛ در غیر این صورت کاربر به تب مربوط، مانند medications یا labs، می‌رود. مقصد فعلی خود تب است و رکورد دقیق را برجسته یا به آن پیمایش نمی‌کند، بنابراین در پرونده شلوغ پیداکردن منبع هنوز زمان می‌برد. لینک سند نیز به احراز هویت Bearer وابسته است و بازشدن آن را در محیط نمایشی حاضر آزمایش نکردم.

۷ـ۶ تصمیم‌های رابط برای قابل پیگیری ماندن پاسخ

برای هر پاسخ دستیار، متن را از فهرست منابع جدا نمایش دادم تا کاربر ابتدا پاسخ را بخواند و در صورت نیاز به رکوردهای استفاده‌شده برگردد. منبع دارویی به تب دارو، نتیجه آزمایش به تب آزمایش و سند به فایل یا بخش اسناد متصل می‌شود. این پیوندها برای بررسی منشأ پاسخ مفیدند، اما هنوز کاربر را دقیقاً روی همان ردیف نمی‌برند. برای کامل‌کردن این مسیر می‌توان شناسه رکورد را در query یا fragment نشانی قرار داد و جزء مقصد پس از بارگذاری، ردیف را برجسته و به آن پیمایش کند. این تغییر باید با کنترل دسترسی فعلی سازگار بماند و نباید شناسه یک بیمار دیگر را فقط بر پایه URL نمایش دهد.

در زمان طراحی پنل، حالت‌های «گفت‌وگوی انتخاب‌شده»، «ارسال درحال انجام» و «نتیجه ایمنی دارو» را مستقل نگه داشتم. این جداسازی مانع می‌شود خطای تحلیل دارو کل گفت‌وگو را از کار بیندازد، اما یک خطر دیگر دارد: کاربر می‌تواند متن دارو را عوض کند و نتیجه قبلی همچنان دیده شود. در نسخه بعدی، نتیجه باید همراه نسخه‌ای از ورودی ذخیره شود و با هر تغییر نام یا دوز، نشان «نتیجه مربوط به ورودی قبلی» بگیرد یا پاک شود. همین قاعده برای پاسخ چت نیز اهمیت دارد؛ نمایش زمان، نام بیمار و منبع باید به کاربر کمک کند بفهمد پاسخ مربوط به کدام زمینه بوده است، نه اینکه صرفاً یک متن تولیدشده در پنل باقی بماند.

فصل ۸ یادداشت صوتی و بررسی ایمنی دارو

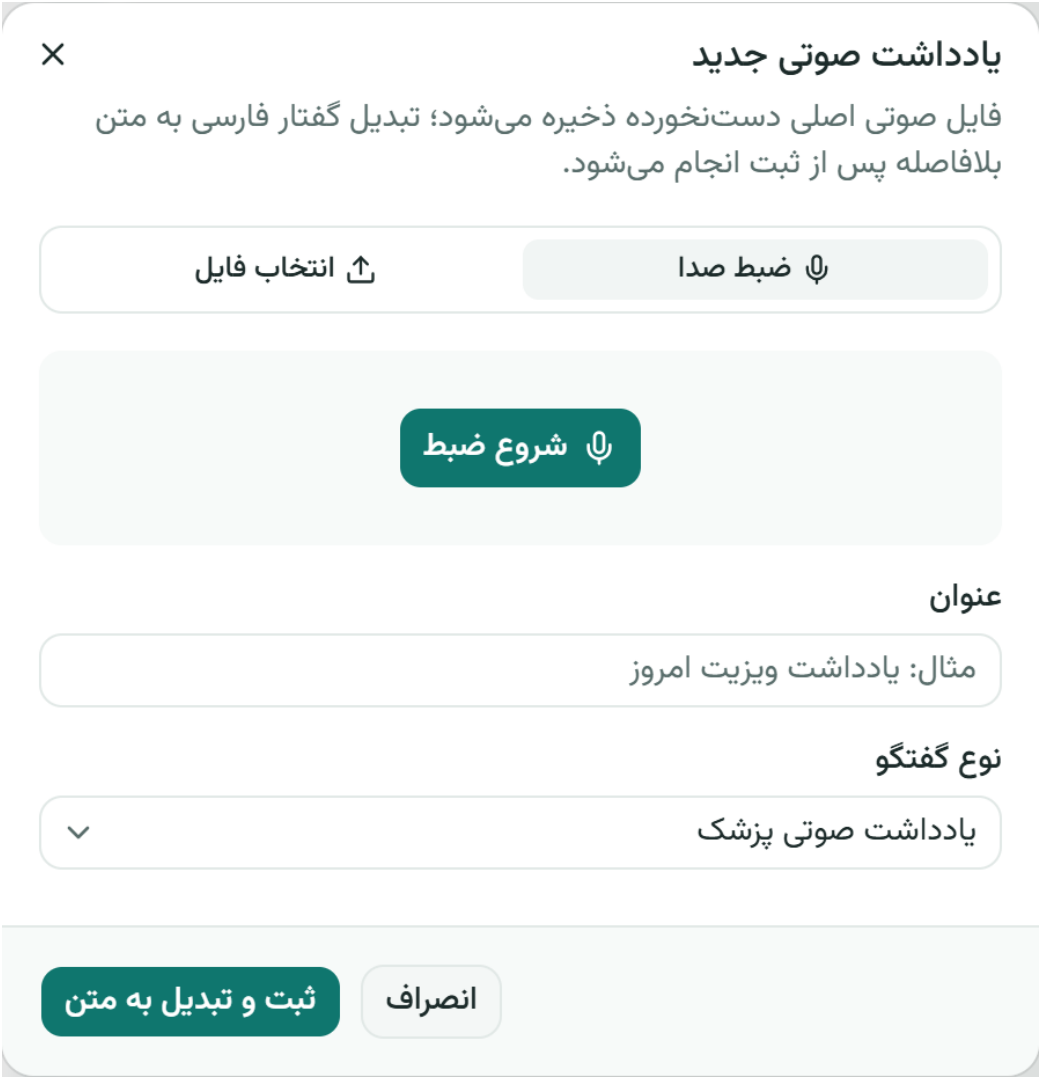
۸ـ۱ دو سناریو در یک گردش کار

AddVoiceNoteButton یک پنجره مشترک برای ضبط یا انتخاب فایل فراهم می‌کند. نوع گفتگو با دو مقدار physician_note و doctor_patient تعیین می‌شود. اولی برای گفتار تک‌نفره و دومی برای مکالمه کنترل‌شده است. انتخاب این مقدار در FormData به سرور فرستاده می‌شود و به سیاست رونویسی مربوط می‌شود. رابط جداگانه‌ای برای تخصیص صریح نقش گوینده به پزشک یا بیمار در کد بررسی‌شده دیده نمی‌شود. بنابراین حتی وقتی سرویس برچسب گوینده برمی‌گرداند، باید آن را برچسب صوتی ناشناس دانست، نه تأیید هویت یا نقش درمانی.

برنامه از سرویس آماده تبدیل گفتار به متن استفاده می‌کند و مدل گفتار در این پروژه آموزش داده نشده است. در فرانت‌اند، کار من دریافت فایل یا ضبط کاربر، نمایش وضعیت پردازش و فراهم‌کردن مرحله بازبینی بود. کیفیت رونویسی، واژه‌های تخصصی و fallback در سرور مدیریت می‌شوند. stt.py علاوه بر Speechmatics مسیرهای Google و OpenRouter را هم دارد، اما این ارائه‌دهنده‌ها توان یکسانی برای تفکیک گوینده ندارند. به همین دلیل، انتخاب «گفت‌وگوی پزشک و بیمار» همیشه به معنای تولید متن نقش‌دار نیست و متن رابط باید با ارائه‌دهنده فعال هماهنگ بماند.

۸ـ۲ انتخاب حالت و مدیریت ورودی

شکل ۱۵ پنجره ثبت صوت را با دو حالت «ضبط صدا» و «انتخاب فایل» نشان می‌دهد. عنوان و نوع گفت‌وگو پیش از ارسال در همان پنجره انتخاب می‌شوند. در اجرای نمایشی اجازه میکروفون ندادم و فایلی هم ارسال نکردم؛ بنابراین این تصویر فقط چیدمان و کنترل‌های ورودی را پوشش می‌دهد و نتیجه رونویسی را نشان نمی‌دهد.



شکل ۱۵ ـ پنجره‌ی واقعی ضبط یا انتخاب یادداشت صوتی

mode در state بین record و file تغییر می‌کند. در حالت ضبط، دکمه شروع یا پایان و شمارنده زمان نشان داده می‌شود. در حالت فایل، کنترل انتخاب فایل با accept صوتی وجود دارد. عنوان اختیاری و نوع گفتگو در هر دو حالت مشترک‌اند. تغییر حالت، قصد ارسال را تغییر می‌دهد؛ handleSubmit در حالت ضبط از recordedRef و در حالت فایل از file استفاده می‌کند. این جداسازی مانع آن است که آخرین فایل انتخاب‌شده بی‌اختیار به جای ضبط تازه ارسال شود. بااین‌حال، رفتار تغییر حالت در حین ضبط باید در مرورگر آزموده شود، زیرا کنترل‌های حالت در نسخه فعلی به وضعیت recording وابسته نشده‌اند.

اگر دسترسی میکروفون داده نشود یا ضبط آغاز نشود، پیام فارسی نمایش داده می‌شود و کاربر می‌تواند فایل صوتی را انتخاب کند. پیام فعلی چند علت متفاوت، از جمله رد اجازه، نبود دستگاه و محیط ناامن، را در یک متن جمع می‌کند. برای MVP این رفتار ساده بود، ولی در نسخه بعدی بهتر است علت‌های قابل تشخیص به راهنمای جداگانه تبدیل شوند. در بررسی حاضر از صدای بیمار استفاده نکردم و ضبط را روی چند مرورگر مختلف آزمایش نکردم.

۸ـ۳ آغاز ضبط و سازگاری نوع رسانه

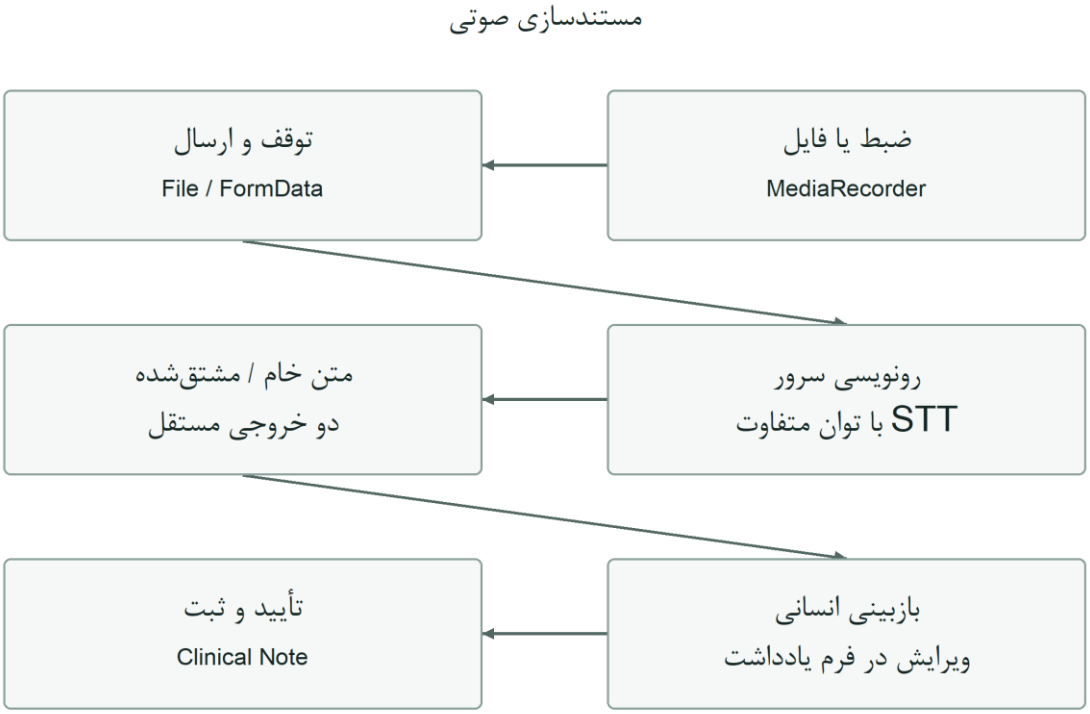
supportedMimeType سه گزینه audio/webm;codecs=opus، audio/webm و audio/mp۴ را به ترتیب با MediaRecorder.isTypeSupported بررسی می‌کند. اگر هیچ‌کدام پشتیبانی نشوند، رشته خالی برمی‌گردد و ساخت recorder بدون گزینه MIME انجام می‌شود. ابتدا getUserMedia با audio:true درخواست stream می‌کند و سپس MediaRecorder ساخته می‌شود. در تعریف این APIها، MediaRecorder رابط ضبط stream رسانه است و getUserMedia به اجازه دسترسی و محیط امن نیاز دارد. در پروژه، انتخاب قابلیت مرورگر انجام شده، اما ضبط موفق و تطابق قالب واقعی فایل همچنان باید عملاً بررسی شوند.

در ondataavailable، قطعه‌های غیرخالی به chunksRef اضافه می‌شوند. recorder.start بدون timeslice اجرا می‌شود و فایل نهایی در onstop از قطعات ساخته می‌شود. این مسیر ارسال پیوسته صوت به API نیست؛ قطعات ابتدا در مرورگر جمع می‌شوند و بعد یک File به سرور می‌رود. نام پسوند بر حسب وجود mp۴ در MIME انتخابی تعیین می‌شود و بخش codecs از نوع multipart حذف می‌شود. اگر MIME انتخابی خالی باشد، فرض پیش‌فرض audio/webm در ساخت Blob به کار می‌رود؛ بررسی سازگاری این فرض با خروجی واقعی مرورگر، یکی از موارد آزمون آینده است.

۸ـ۴ زمان، توقف و آزادسازی میکروفون

تایمر هر ثانیه seconds را افزایش می‌دهد و مقدار آن در دکمه پایان ضبط نشان داده می‌شود. recorder.stop، stopRecording و stopTimer را اجرا می‌کند و state recording را false می‌گذارد. در onstop، تمام trackهای stream متوقف و فایل ساخته می‌شود. این آزادسازی مهم است، زیرا پایان ظاهر ضبط نباید به فعال‌ماندن نامحدود میکروفون منجر شود. همچنین هنگام بسته‌شدن پنجره، cleanup مربوط به open تایمر و recorder را متوقف می‌کند. مسیر فعلی، چند منبع را کنترل می‌کند، ولی کامل‌بودن پاک‌سازی در همه خطاهای میان دریافت stream و ساخت recorder باید آزمایش شود.

هنگام خواندن کد ضبط متوجه شدم callback مربوط به onstop مقدار seconds را از closure زمان شروع می‌خواند. این مقدار ممکن است با شمارنده آخرین رندر برابر نباشد و مدت قدیمی در metadata فایل ثبت شود. از طرف دیگر، تغییر recordedRef به‌تنهایی باعث رندر نمی‌شود و ترتیب اجرای onstop و تغییر recording می‌تواند روی نمایش آماده‌شدن فایل اثر بگذارد. این دو حالت را در مرورگر بازتولید نکردم، اما ساختار state و ref احتمال آن را ایجاد می‌کند. راه‌حل مناسب، ثبت زمان شروع و پایان مستقل از closure و نگهداری وضعیت صریح برای آماده‌شدن فایل است.



شکل ۱۶ ـ ثبت صوت، رونویسی و تأیید یادداشت بالینی

۸ـ۵ ارسال و وضعیت پردازش

در handleSubmit، اگر فایل نهایی موجود نباشد پیام «اول ضبط کنید» یا «انتخاب فایل صوتی الزامی است» نمایش داده می‌شود. uploadVoiceNote، file و در صورت وجود conversation_type، title و duration_seconds را به FormData اضافه می‌کند. مدت به عدد صحیح گرد می‌شود. در حالت انتخاب فایل، مدت از رابط ارسال نمی‌شود و مقدار صفر فرضی به‌عنوان مدت واقعی استفاده نمی‌شود. سرور فایل اصلی را نگه می‌دارد و پردازش رونویسی در پس‌زمینه آغاز می‌شود. حداکثر حجم پذیرفته‌شده در مسیر بک‌اند ۵۰ مگابایت است و از محدودیت سند جداست.

TranscriptionStatus مانند استخراج سند چهار حالت دارد. نمای پرونده badge مربوط را نشان می‌دهد و در شکست، error_message در صورت وجود ارائه می‌شود. برای completed، کنترل ساخت یادداشت ظاهر می‌شود و برای failed، تکرار رونویسی در دسترس است. وضعیت cleanup جداست و نشان «پاک‌سازی شد» فقط در completed آن ظاهر می‌شود. این تفکیک به معنی آن است که رونویسی می‌تواند موفق باشد ولی پاک‌سازی AI انجام نشده یا ناموفق باشد. نباید شکست مرحله دوم به عنوان فقدان کل رونوشت تعبیر شود یا متن خام از دید کاربر پنهان گردد.

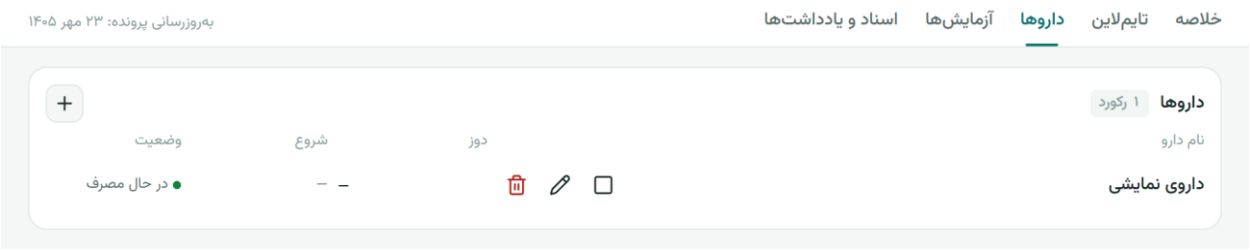
۸ـ۶ متن خام، نسخه مشتق‌شده و فرم بازبینی

اگر transcript موجود باشد، ViewTranscriptButton متن خام را در پنجره فقط‌خواندنی باز می‌کند. ViewCleanedNoteButton نیز cleaned_note را همراه توضیح منشأ هوش مصنوعی نشان می‌دهد. CreateNoteFromVoiceDialog ابتدا cleaned_note و در نبود آن transcript را داخل فرم می‌گذارد. این ترتیب مرور متن مرتب‌شده را آسان می‌کند، ولی لازم است کاربر بداند متن اولیه از کدام نسخه آمده است. در نسخه فعلی مقایسه هم‌زمان متن خام و پاک‌سازی‌شده داخل فرم وجود ندارد.

فرم بازبینی title و content را می‌فرستد و ذخیره فقط با کنش کاربر انجام می‌شود. ویژگی required جلوی ارسال متن کاملاً خالی را در مرورگر می‌گیرد، اما دقت بالینی یا کامل‌بودن محتوا را نمی‌سنجد. متن پنجره از کاربر می‌خواهد پیش از ثبت، یادداشت را مرور کند؛ برنامه نمی‌تواند انجام واقعی این بازبینی را تضمین کند. در نسخه بعدی می‌توان پیوند صوت اصلی، منشأ متن و تغییرهای کاربر را کنار یادداشت نگه داشت. این قابلیت‌ها در نسخه فعلی وجود ندارند و به قرارداد دقیق‌تر و بررسی حریم خصوصی نیاز دارند.

۸ـ۷ هدف و تفاوت با تجویز

شکل ۱۷ ردیف «داروی نمایشی» را در تب داروهای پرونده ساختگی نشان می‌دهد. برای این آزمون، دوز و تاریخ شروع را خالی گذاشتم و وضعیت پیش‌فرض «در حال مصرف» نمایش داده شد. هدف از رکورد، بررسی ارتباط فرم با فهرست و تایم‌لاین بود و از آن برای توصیه مصرف یا ارزیابی ایمنی بالینی استفاده نشد.



شکل ۱۷ ـ ردیف داروی نمایشی پس از ثبت از فرم برنامه

بخش ایمنی دارو یک تحلیل کمکی برای داروی مورد نظر ارائه می‌کند و نقش تجویز یا صدور مجوز مصرف ندارد. فرم نام دارو و در صورت وجود دوز را برای تحلیل می‌فرستد و نتیجه را کنار همان ورودی نشان می‌دهد. در متن رابط نوشته‌ام که تحلیل اختیاری است و جلوی ذخیره رکورد را نمی‌گیرد. به این ترتیب تصمیم نهایی خارج از مدل باقی می‌ماند. نبود هشدار نیز به‌معنای ایمن‌بودن قطعی دارو نیست، چون نتیجه به اطلاعات موجود پرونده و محدودیت مدل وابسته است.

در پروپوزال، هشدارهای منتخب بر پایه قواعد بودند و مدل زبانی بیشتر نقش توضیح را داشت. مخزن فعلی هم موتور قواعد مستقل دارد و هم مسیر AI-assisted، اما MedicationFormDialog مستقیماً مسیر AI-assisted را فراخوانی می‌کند. بنابراین هشدار این فرم را نباید صرفاً به medication_rules.json نسبت

داد. در جدول ۳ این دو مسیر را جدا کرده‌ام تا تغییر رفتار نسخه فعلی نسبت به طرح اولیه و کاری که رابط واقعاً نمایش می‌دهد روشن باشد.

جدول ۳ ـ تفاوت مسیر قواعد و تحلیل AI در رابط دارو

ویژگی	مسیر قواعد	مسیر AI در فرم
endpoint	medications/check/	medications/ai-safety-check/
منطق	قواعد و داده محلی منتخب	تحلیل مدل با زمینه بیمار
خروجی رابط جاری	درخواست از این فرم دیده نشد	overall_assessment و آرایه‌های متن
شدت/منبع	قرارداد مستقل قواعد	severity و rule_id در نوع AI نیست
اثر بر ذخیره	مسیر جدا؛ معادل تأیید جامع نیست	اختیاری؛ ذخیره را مسدود نمی‌کند
وضعیت ارزیابی	اعتبار پزشکی جامع اثبات نشده	حساسیت/ویژگی بالینی اندازه‌گیری نشده

۸ـ۸ اجرای تحلیل و استقلال وضعیت‌ها

در شکل ۱۸ حالت «بررسی ایمنی دارو» در پنل دستیار فعال است و متن راهنما از کاربر می‌خواهد نام داروی مورد نظر را همراه پرسش بنویسد. هنگام گرفتن تصویر فقط وضعیت رابط را تغییر دادم و درخواستی به مدل نفرستادم. شکل نقطه ورود به تحلیل را نشان می‌دهد، نه نتیجه یا تأیید پزشکی آن را.

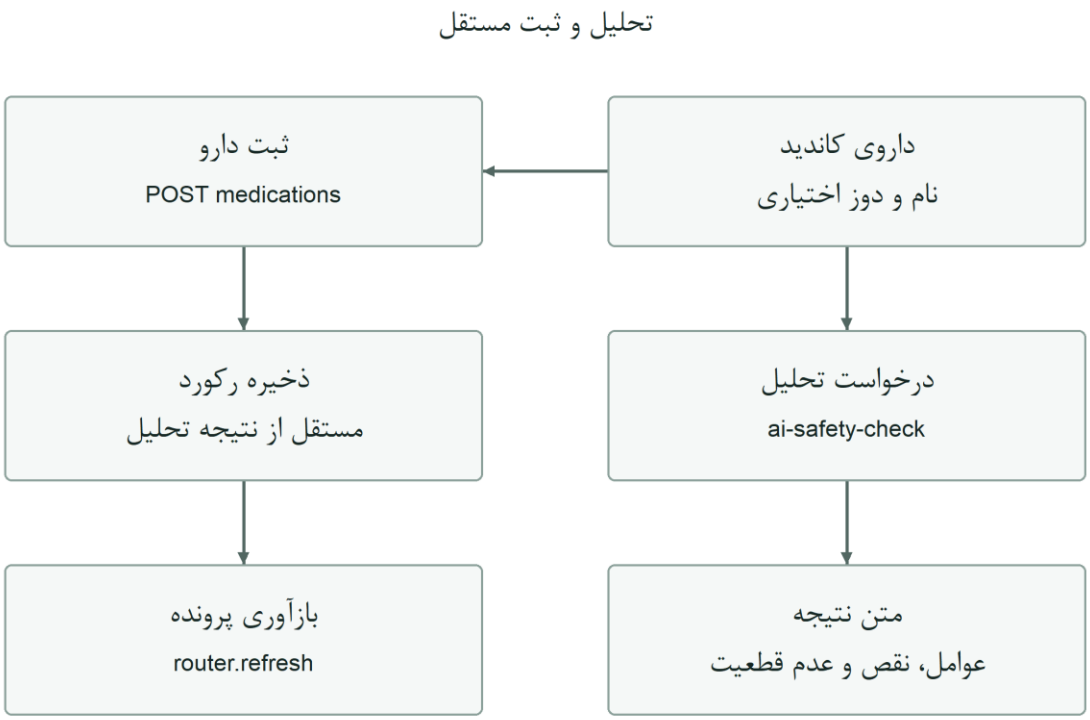


شکل ۱۸ ـ کنترل‌های حالت ایمنی دارو در پنل گفت‌وگو

runSafetyCheck نام را trim می‌کند و اگر نام خالی یا درخواست درحال انجام باشد، برمی‌گردد. safetyError، safetyLoading و safetyResult از submitting و error ذخیره جدا هستند. درخواست شامل medications با یک عنصر نام و dose است، هرچند قرارداد API در سرور تعداد بیشتری را اجازه می‌دهد. داشتن ظرفیت چند دارو در API به معنای وجود کنترل چندکاندیدی در این فرم نیست. پس

از پاسخ، نتیجه ذخیره می‌شود؛ در شکست، خطای تحلیل جدا ارائه می‌شود و فرم ثبت همچنان موجود می‌ماند. این تفکیک وضعیت از آمیختن شکست AI و شکست ثبت رکورد جلوگیری می‌کند.

handleSubmit مستقل از نتیجه تحلیل، createPatientMedication یا updatePatientMedication را اجرا می‌کند. دکمه ذخیره به submitting وابسته است و safetyLoading یا شدت نتیجه آن را متوقف نمی‌کند؛ در نتیجه کاربر می‌تواند پیش از پایان تحلیل یا بعد از خطای آن رکورد را ثبت کند. این جداسازی را برای جلوگیری از یکی‌گرفتن «ثبت اطلاعات» با «تأیید ایمنی» حفظ کردم، اما متن و وضعیت رابط هنوز باید این تفاوت را واضح‌تر نشان دهند. مناسب‌بودن این رفتار در محیط درمانی به سیاست بالینی و ارزیابی جداگانه نیاز دارد.



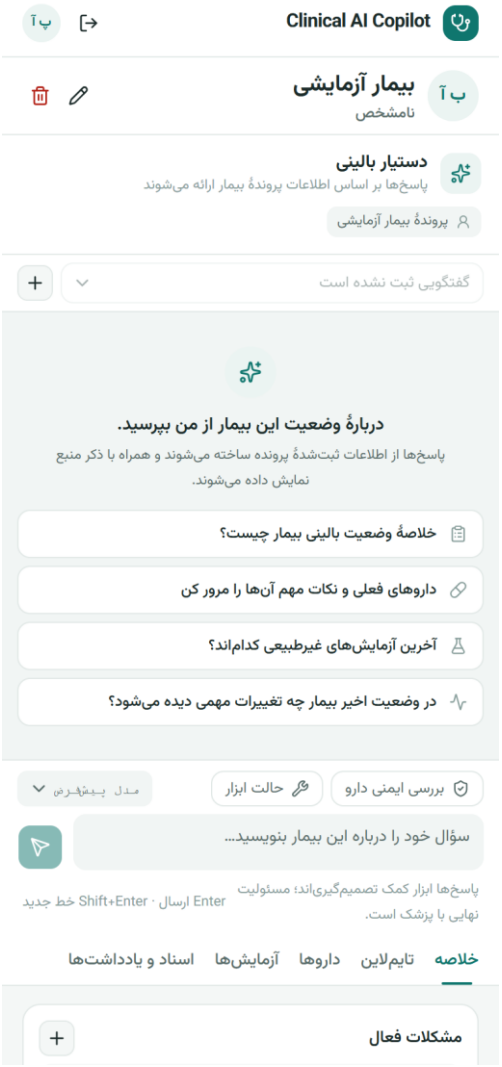
شکل ۱۹ ـ استقلال تحلیل ایمنی از ثبت رکورد دارویی

فصل ۹ رابط فارسی و طراحی واکنش‌گرا

۹ـ۱ مشاهده محدود چیدمان موبایل

نمای موبایل را با عرض ۳۹۰ پیکسل در مرورگر محلی بررسی کردم. در صفحه معرفی، عناصر اصلی عمودی شدند و فرم ورود بریدگی افقی نداشت. پس از راه‌اندازی پایگاه داده آزمایشی، فضای پرونده را هم در همین عرض باز کردم: پنل گفت‌وگو پیش از کارت‌های خلاصه قرار گرفت، نوار تب‌ها قابل پیمایش بود و عرض صفحه از viewport بیشتر نشد. شکل ۲۰ و شکل ۲۱ نتیجه همین بررسی را نشان می‌دهند. این آزمون روی یک اندازه و یک مرورگر انجام شد و پوشش همه گوشی‌ها یا فناوری‌های کمکی نیست.

در شکل ۲۰ نام بیمار، پنل دستیار و ناوبری پرونده به‌ترتیب عمودی دیده می‌شوند. شکل ۲۱ نیز چیدمان تک‌ستونی کارت‌ها و داروی نمایشی را در همان پرونده نشان می‌دهد. در این اجرا اسکرول افقی ناخواسته ندیدم، اما حالت بزرگ‌نمایی و اندازه‌های دیگر را جداگانه آزمایش نکردم.



شکل ۲۰ ـ بخش آغازین فضای پرونده در نمای موبایل



شکل ۲۱ ـ کارت‌های خلاصه‌ی پرونده در چیدمان تک‌ستونی موبایل

هر دو تصویر مستقیماً از صفحه اجراشده و داده ساختگی همان پرونده گرفته شده‌اند.

۹ـ۲ وضوح کنش و پیشگیری از خطا

برای هدایت ورود داده، فرم‌ها عنوان، توضیح، Label و ویژگی required دارند. متن دکمه هنگام ارسال تغییر می‌کند و در مسیرهای لازم غیرفعال می‌شود. ConfirmDialog نیز پیش از حذف سند، گفت‌وگو یا رکورد، پیامد کار را توضیح می‌دهد؛ در قطع دارو هم مشخص است که سابقه حذف نمی‌شود. با وجود این، بعضی آیکون‌ها ممکن است انتظار متفاوتی بسازند؛ برای نمونه آیکون مداد کنار مشاهده متن استخراج‌شده بیشتر یادآور ویرایش است. در آزمون کاربری بعدی باید انتظار کاربر از آیکون و نتیجه واقعی هر کنش ثبت شود.

پرکردن فرم از رکورد موجود و نگه‌داشتن مقدار در خطا، از ورود تکراری جلوگیری می‌کند. ولی بستن پنجره درحال ضبط یا انتظار، تغییر حالت فایل و ثبت نتیجه تحلیل قدیمی مثال‌هایی‌اند که پیشگیری کامل به منطق بیشتر نیاز دارد. برای طراحی بهتر، باید برای هر فرم حالت‌های مجاز و کنش‌های هر حالت مشخص شود. این

الگوی حالت، هم به آزمون کمک می‌کند و هم ناسازگاری ظاهر با عملیات را کم می‌کند. وجود چند boolean مستقل همیشه وضعیت‌های نامعتبر را غیرممکن نمی‌کند؛ در برخی زیرسامانه‌ها یک مدل حالت صریح می‌تواند قرارداد را روشن‌تر سازد.

۹ـ۳ اعلام وضعیت و بازیابی شکست

در حالت انتظار چت از aria-live=polite استفاده کرده‌ام تا متن «در حال بررسی پرونده» بدون جابه‌جایی تمرکز اعلام شود. در فرم‌های دیگر، خطا بیشتر به‌صورت یک پاراگراف رنگی نمایش داده می‌شود و همه آن‌ها live region ندارند. بنابراین پوشش دسترس‌پذیری پیام‌ها یکدست نیست. لازم است خطاهای آپلود و ثبت با صفحه‌کلید و صفحه‌خوان آزمایش شوند و پیام‌های مهم به ناحیه قابل اعلان منتقل شوند.

بازیابی شکست شامل حفظ ورودی، توضیح علت امن و راه اقدام است. پیام عمومی «داده‌های ارسالی معتبر نیست» برای خطای یک فیلد یا اندازه فایل، اقدام دقیق را روشن نمی‌کند. در پردازش سند و صوت، دکمه retry برای failed وجود دارد، اما تغییر completed باید با بازخوانی دیده شود. در چت، پرسش ناموفق دوباره در ورودی قرار می‌گیرد، ولی اثر ذخیره‌شدن قبلی سرور روشن نیست. این مثال‌ها نشان می‌دهند «نمایش خطا» با «بازیابی قابل پیش‌بینی» تفاوت دارد. ارزیابی باید ببیند کاربر بعد از خطا چه کنشی انجام می‌دهد و آیا داده تکراری یا رهاشده ایجاد می‌شود.

۹ـ۴ صفحه‌کلید و تمرکز دیداری

در CSS برای لینک، دکمه، ورودی، select، textarea و عناصر دارای tabindex قاعده focus-visible تعریف کردم و outline از توکن مشترک استفاده می‌کند. این قاعده محل تمرکز را برای کار با صفحه‌کلید قابل مشاهده می‌کند. با این حال، باید در پس‌زمینه‌های مختلف، دکمه‌های کوچک، پنجره‌ها و ناحیه sticky هم آزمایش شود؛ چون outline ممکن است داخل ظرف overflow-hidden بریده شود. مرور کامل همه کنترل‌ها با صفحه‌کلید در این مرحله انجام نشد.

Dialogهای عمومی عنوان و در بعضی موارد description دارند و رفتار پایه را از جزء کتابخانه می‌گیرند. با این حال، هر فرم باید از نظر ورود فوکوس، برگشت فوکوس پس از بستن، حرکت میان کنترل‌ها و خروج با صفحه‌کلید بررسی شود. تب‌های پرونده در پیاده‌سازی فعلی لینک ناوبری هستند. textarea چت placeholder دارد، اما برچسب قابل دسترسی مستقلی برای خود کنترل ندیدم؛ aria-label ظرف بیرونی جای نام textarea را نمی‌گیرد. افزودن برچسب مشخص برای این ورودی یکی از اصلاح‌های لازم است.

فصل ۱۰ آزمون و تحویل سامانه

۱۰ـ۱ سطوح متفاوت راستی‌آزمایی

برای بررسی نسخه نهایی سه کار انجام دادم: مسیرهای مهم کد را خواندم، ابزارهای توسعه را اجرا کردم و چند گردش کار را در مرورگر دیدم. خواندن کد قرارداد تابع‌ها را روشن می‌کند؛ TypeScript و build خطاهای نوع و ساخت را می‌سنجند؛ و مرورگر برای رفتار واقعی ناوبری، فرم و شبکه لازم است. هیچ‌یک از این مراحل جای ارزیابی بالینی یا آزمون کاربردپذیری با کاربر را نمی‌گیرد. چون بخش اصلی کار من با تعامل کاربر ارتباط داشت، نتیجه build را به‌تنهایی معیار کیفیت رابط در نظر نگرفتم.

برای این بررسی از وابستگی‌های نصب‌شده پروژه استفاده کردم و پیش از اجرای کنترل‌ها کد را برای گرفتن نتیجه بهتر دست‌کاری نکردم. TypeScript را با noEmit و بدون lint، incremental را بدون fix و سپس ساخت تولیدی را اجرا کردم. برای گرفتن تصاویر بخش خصوصی، یک PostgreSQL جداگانه روی پورت آزمایشی راه انداختم، مهاجرت‌های مخزن را اجرا کردم و API را به نشانی تنظیم‌شده فرانت‌اند متصل کردم. همه حساب‌ها و رکوردها ساختگی بودند و هیچ کلید، گذرواژه یا داده واقعی بیمار وارد گزارش نشد.

۱۰ـ۲ نتیجه اجرای جاری فرانت‌اند

TypeScript با کد خروج صفر تمام شد. ESLint کد خروج یک داد و ۷ خطا و ۲ هشدار ثبت کرد. بیشتر خطاها مربوط به set-state-in-effect در فرم‌های حساسیت، شرایط، سند، دارو، بیمار و دو بخش صوت بودند. دو هشدار نیز به دستور غیرفعال‌سازی بدون استفاده و هدایت با window.location.href در api.ts مربوط می‌شدند. این نتیجه نشان داد الگوی فعلی reset بعضی فرم‌ها با قانون lint پروژه سازگار نیست. خطاها به معنی ازکارافتادن همه فرم‌ها نیستند، اما تا زمان اصلاح آن‌ها نمی‌توان lint را موفق اعلام کرد. جزئیات سه دستور در جدول ۴ آمده است.

جدول ۴ ـ نتایج بررسی‌های اجراشده در نسخه جاری

بررسی	نتیجه جاری	دامنه
TypeScript noEmit	موفق؛ exit=۰؛ ۵.۷۸ثانیه	سازگاری نوع؛ نه آزمون تعامل
ESLint بدون fix	ناموفق؛ exit=۱؛ ۷خطا و۲هشدار	قواعد کیفیت نصب‌شده
Next production build	موفق؛ exit=۰؛ ۵۱.۰۱ثانیه	ساخت برنامه؛ نه استقرار یا E۲E
API و مرورگر محلی	مهاجرت موفق؛ ثبت پزشک، بیمار و داروی نمایشی مشاهده شد	گردش محدود؛ نه آزمون جامع API یا AI

بررسی	نتیجه جاری	دامنه
مطالعه کاربر و SUS	اجراشده یافت نشد	پروتکل آینده؛ امتیاز ساخته نشد

ساخت تولیدی با Next.js ۱۶٫۳٫۲ و Turbopack موفق شد و دو مسیر بیمار به‌صورت پویا در خروجی گزارش شدند. اجرای این دستور در همان سیستم حدود ۵۱ ثانیه طول کشید؛ این عدد فقط زمان یک build است و معیار کارایی API یا تجربه کاربر نیست. صفحات معرفی، ورود و not-found به‌صورت ایستا ساخته شدند. این نتیجه نشان می‌دهد بسته تولیدی در محیط فعلی ساخته می‌شود، اما پخش فایل، ضبط میکروفون و همه درخواست‌های API داخل دستور build اجرا نمی‌شوند.

۱۰ـ۳ سناریوهای یکپارچه‌سازی

در محیط نمایشی، ثبت حساب آزمایشی، ایجاد بیمار، نمایش فهرست، جابه‌جایی تب‌ها، ثبت داروی نمایشی و پیدایش رویداد آن در تایم‌لاین را اجرا کردم. پنجره‌های سند و صوت و حالت ایمنی دارو نیز باز شدند. در این سناریو فایل نفرستادم، ضبط واقعی انجام ندادم و درخواستی به مدل ارسال نکردم. در جدول ۵ برای هر مسیر دقیقاً همان مرحله‌ای را نوشته‌ام که اجرا شد تا نتیجه چند کنش ساده به کل زنجیره تعمیم داده نشود.

جدول ۵ ـ سناریوهای یکپارچه‌سازی و سطح شاهد

گردش	شاهد ساختاری	شاهد مرورگر / باقی‌مانده
ثبت بیمار	payload، API، refresh	ثبت و فهرست مشاهده شد؛ خطاها باقی
دارو و تایم‌لاین	ثبت با API و تولید رویداد	هر دو در مرورگر دیده شد؛ دقت بالینی آزمون نشد
سند	FormData و وضعیت استخراج	پنجره دیده شد؛ آپلود و OCR اجرا نشد
صوت	File و مسیر ثبت یادداشت	پنجره دیده شد؛ ضبط و STT اجرا نشد
گفتگو	تاریخچه و مدیریت پاسخ	حالت خالی دیده شد؛ مدل فراخوانی نشد
ایمنی دارو	حالت ابزار و تحلیل اختیاری	حالت رابط دیده شد؛ تحلیل AI اجرا نشد
مجوز بیمار	کنترل مسیر بیمار و منابع	پرونده خود پزشک دیده شد؛ نقش دیگر آزمون نشد

مسیر موفق همه ابهام‌ها را نشان نمی‌دهد؛ برای همین سناریوهای قطع ارتباط هنگام ارسال، ۴۰۱ پس از انقضای نشست، ۴۰۳ برای بیمار غیرمجاز، فایل بزرگ، متن خالی و پاسخ کند باید جداگانه اجرا شوند. در هر

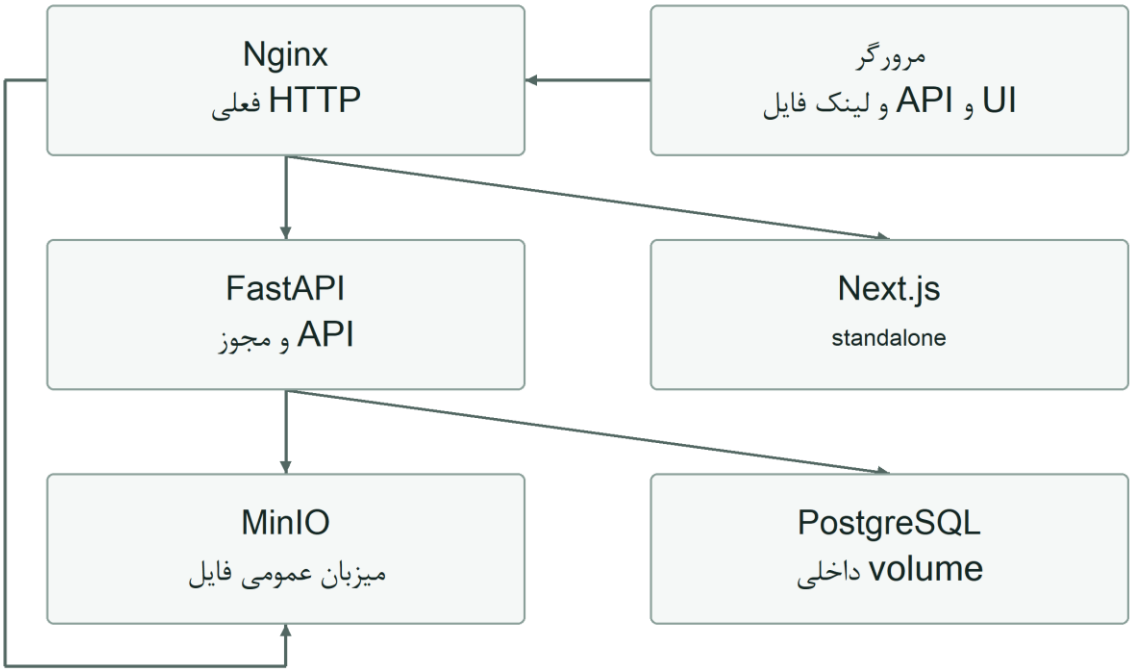
سناریو باید مشخص باشد عملیات پیش از خطا چه اثری گذاشته است. نمونه مهم، چت است که ممکن است پرسش را در سرور ثبت کند ولی رابط همان متن را به ورودی برگرداند. این سناریوها را از روی کد استخراج کردم، اما همه آن‌ها در مرورگر اجرا نشدند. برای اجرای تکرارپذیر باید داده ساختگی و پیش‌شرط ثابت تعریف شود.

۱۰ـ۴ توپولوژی سرویس و فایل اصلی

Compose تولیدی سرویس‌های PostgreSQL، backend، frontend، Nginx و MinIO را در یک شبکه قرار می‌دهد. پورت‌های داخلی با expose تعریف شده‌اند و فقط Nginx پورت میزبان را منتشر می‌کند. داده پایگاه و فایل‌ها نیز volume پایدار دارند. backend تا سالم‌شدن PostgreSQL و MinIO منتظر می‌ماند و healthcheck خودش را دارد. در مقابل، depends_on فرانت‌اند برای backend فقط service_started است و آماده‌بودن کامل API یا اجرای migration را تضمین نمی‌کند. migration در این پیکربندی یک مرحله دستی است. برای تهیه گزارش، استقرار تولیدی تازه‌ای انجام ندادم.

برای URLهای امضاشده، MINIO_ENDPOINT باید میزبان عمومی فایل را منعکس کند تا مرورگر بتواند آن را باز کند. Compose میزبان storage را به host-gateway برای backend نگاشت می‌کند و Nginx همان Host را به MinIO می‌فرستد. تغییر Host می‌تواند امضای درخواست فایل را ناسازگار کند؛ توضیح این الزام در nginx.conf موجود است. از دید فرانت‌اند، لینک فایل به دو شرط نیاز دارد: احراز مرحله API و دسترسی شبکه به میزبان فایل. حل شرط دوم در توپولوژی، شرط نخست را خودکار حل نمی‌کند. شکل ۲۲ این دو مرز را در معماری تحویل نشان می‌دهد.

توپولوژی پیکربندی



شکل ۲۲ ـ توپولوژی پیکربندی تحویل و مسیر فایل

۵ـ۱۰ پروکسی، زمان انتظار و حد حجم

Nginx مسیر /api/ را به backend و مسیرهای دیگر را به frontend می‌فرستد و /health نیز جدا تعریف شده است. timeout عملیات طولانی API برابر ۳۰۰ ثانیه است، اما این مقدار زمان واقعی پاسخ مدل یا timeout مرورگر را تعیین نمی‌کند. client_max_body_size روی ۲۵m قرار دارد، در حالی که بک‌اند فایل صوتی را تا ۵۰ مگابایت می‌پذیرد. در نتیجه فایل بزرگ‌تر از حد Nginx پیش از رسیدن به کنترل بک‌اند رد می‌شود. این دو مقدار باید در پیکربندی نهایی هماهنگ شوند.

Compose پورت ۴۴۳ را منتشر می‌کند، ولی nginx.conf فعلی فقط ۸۰ listen دارد و TLS به مرحله بعد موکول شده است. انتشار پورت به‌تنهایی HTTPS را راه‌اندازی نمی‌کند. این موضوع برای ورود، دریافت فایل، دسترسی میکروفون و clipboard مهم است. در جدول ۶ تنظیمات اثرگذار بر رابط را کنار هم آورده‌ام. برای فعال‌کردن سرویس عمومی باید دامنه و گواهی مناسب تنظیم شوند و مسیر API و فایل هر دو در همان محیط آزمایش شوند؛ چنین استقرار عمومی‌ای در این پروژه انجام نشد.

جدول ۶ ـ تنظیمات تحویل مؤثر بر رفتار رابط

تنظیم	مقدار یا رفتار	اثر رابط

تنظیم	مقدار یا رفتار	اثر رابط
NEXT_PUBLIC_API_URL	آدرس مطلق Compose؛ زمان build می‌خواهد	هر دو browser و RSC باید دسترسی داشته باشند
خروجی Next	standalone	server.js همراه static و public
Nginx /api	backend؛ انتظار ۳۰۰ثانیه	جدا از timeout داخلی درخواست
حجم پروکسی	۲۵m	ممکن است صوت۵۰MB پیش از API رد شود
TLS	پورت۴۴۳ منتشر؛ config فعلی listen۸۰	HTTPS عملی اثبات نشده
فایل	میزبان عمومی storage و Host ثابت	درستی امضا و دسترسی مرورگر
پایگاه/MinIO	volume و healthcheck	وجود config به معنای استقرار موفق نیست

فصل ۱۱ ارزیابی و جمع‌بندی

۱۱ـ۱ نتیجه کار من در نسخه موجود

طبق تقسیم کار پروپوزال، مسئولیت اصلی من معماری و پیاده‌سازی فرانت‌اند بود. در نسخه فعلی، ساختار Next.js، مسیرها، پوسته فارسی، اجزای مشترک و توکن‌های طراحی وجود دارند. فهرست بیمار جست‌وجو و صفحه‌بندی دارد و پرونده مشخصات، شرایط، حساسیت‌ها، داروها، آزمایش‌ها، اسناد و یادداشت‌ها را نمایش می‌دهد. برای سند، مسیر آپلود و مشاهده وضعیت ساخته شده و بخش صوت امکان ضبط یا انتخاب فایل، دیدن رونوشت و ساخت یادداشت پس از بازبینی را فراهم می‌کند. گفت‌وگوی پایدار، انتخاب مکالمه و نمایش منابع داخل پرونده نیز در رابط قرار گرفته‌اند. یکپارچه‌سازی این بخش‌ها با API کار مشترک تیم بود و تاریخچه Git برای نسبت‌دادن همه خطوط به یک نفر جزئیات کافی ندارد.

چند مورد از طرح اولیه هنوز کامل نشده‌اند: اصلاح دستی OCR، پلیر داخلی و پیش‌شنیدن ضبط، نسبت‌دادن نقش پزشک یا بیمار به گوینده، خلاصه مستقل هوش مصنوعی در نمای Summary و نمایش جامع منابع پزشکی بیرونی. مسیر OTP در سرور وجود دارد، اما در صفحه ورود عرضه نشده است. تحلیل ایمنی فرم دارو نیز در نسخه فعلی از مسیر AI-assisted می‌آید و با طرح اولیه قواعد منتخب یکسان نیست. در جدول ۷ وضعیت هر مورد را با سه برچسب کامل، جزئی و نیازمند بررسی آورده‌ام تا فاصله میان سرویس موجود و گردش کار قابل استفاده روشن بماند.

جدول ۷ ـ تطبیق وظایف ایمان با امکانات جاری

وظیفه ایمان در پروپوزال	شواهد موجود	وضعیت و شکاف
معماری و طراحی سیستم	App Router، RTL، shared/ui و توکن‌ها	موجود؛ اجرای انفرادی همه اجزا اثبات نشده
داشبورد و پرونده	فهرست، فرم‌ها، خلاصه و تب‌ها	موجود؛ برخی مسیرهای تاریخچه محدود
اسناد	آپلود، نشان وضعیت و متن	جزئی؛ اصلاح OCR و کنترل تحلیل UI تأیید نشده
صوت و مکالمه	ضبط/فایل، متن، بازبینی و ذخیره	جزئی؛ پخش داخلی و نقش گوینده تأیید نشده
تایم‌لاین و Summary	نمای ساختاریافته و رویدادها	موجود؛ summary AI مستقل نیست
چت و Citation	conversation و evidence داخلی	موجود؛ منبع بیرونی نیست؛ دانلود نیاز اصلاح

وظیفه ایمان در پروپوزال	شواهد موجود	وضعیت و شکاف
نمای هشدار دارو	فرم AI اختیاری	جزئی؛ قرارداد شدت/منبع قواعد در این فرم نیست
Usability و تست	type/lint/build و تحلیل گردش	مطالعه کاربر/SUS و E۲E جاری تأیید نشده
Integration و Deployment	APIها، Docker و Nginx	کار مشترک؛ استقرار زنده تأیید نشده

۱۱ـ۲ محدودیت ارزیابی انجام‌شده

آزمون TypeScript و build موفق شدند و lint با ۷ خطا و ۲ هشدار پایان یافت. این نتیجه‌ها فقط بخشی از کیفیت مهندسی نسخه را نشان می‌دهند. همه APIها روی زیرساخت زنده اجرا نشدند، ضبط روی چند مرورگر آزمایش نشد و ارزیابی با پزشک انجام نگرفت. به همین دلیل عددی برای SUS، زمان مرور پرونده، دقت STT، کاهش خطای درمان یا کیفیت تشخیص ارائه نمی‌کنم. هر کدام از این شاخص‌ها به داده، سناریو و روش ارزیابی جداگانه نیاز دارند.

نتیجه‌های این فصل به نسخه کد و محیط محلی استفاده‌شده محدودند. پاسخ مدل با تغییر داده بیمار، تنظیمات یا ارائه‌دهنده می‌تواند عوض شود. برای تکرار آزمایش‌های بعدی باید نسخه کد، نام مدل، ورودی ساختگی و معیار ارزیابی ثبت شوند. نگهداری متن پاسخ بدون snapshot زمینه نیز برای بازسازی کامل شرایط گذشته کافی نیست. این محدودیت را باید هم در طراحی آزمون و هم در امکان بازبینی پاسخ‌های قدیمی در نظر گرفت.

۱۱ـ۳ اصلاح‌های ضروری رابط

اولین اصلاح، هماهنگ‌کردن قراردادهای رابط و سرور است. دانلود فایل باید از مسیر مجاز Bearer یا proxy انجام شود. در شکست چت باید معلوم باشد پرسش در سرور ثبت شده یا نه، و نتیجه تحلیل دارو باید به همان نام و دوز ورودی متصل بماند. در سند و صوت نیز «دریافت درخواست» باید از «پایان پردازش» جدا نمایش داده شود. بخشی از این تغییرها در فرانت‌اند انجام می‌شوند و بخشی به پاسخ یا metadata تازه از بک‌اند نیاز دارند.

پس از آن، پاک‌سازی منابع ضبط، محاسبه مدت و حالت آماده‌بودن فایل باید در چند مرورگر آزمایش شوند. نمایش شرایط تاریخی، خطاهای وابسته به فیلد و برچسب قابل دسترسی ورودی چت نیز کامل نیستند. برای رفع lint بهتر است الگوی reset فرم اصلاح شود و قانون صرفاً خاموش نشود. در استقرار هم حد حجم صوت،

نشانی API و TLS باید با یکدیگر هماهنگ شوند. هر اصلاح باید با آزمونی که همان مشکل را پوشش می‌دهد دوباره سنجیده شود.

۱۱ـ۴ آنچه در مرورگر اجرا کردم

صفحه‌های عمومی و بخش‌های نیازمند ورود را با نسخه تولیدی Next.js در مرورگر باز کردم. برای بخش خصوصی، PostgreSQL جداگانه را با داده ساختگی و مهاجرت‌های مخزن راه انداختم. ثبت‌نام، ایجاد بیمار، ثبت یک داروی نمایشی و نمایش آن در خط زمانی اجرا شدند. بارگذاری سند، رونویسی صوت، تولید پاسخ مدل، دانلود فایل و تحلیل دارو را در این سناریو اجرا نکردم. تصاویر این بخش‌ها فقط پنجره و کنترل‌های رابط را نشان می‌دهند و نتیجه پردازش سرور را تأیید نمی‌کنند.

۱۱ـ۵ ادامه آزمون یکپارچه

محیط نمایشی برای مسیرهای ساده ثبت و مرور آماده است. مرحله بعد باید ذخیره‌سازی فایل و ارائه‌دهنده‌های لازم برای سند، صوت و گفت‌وگو را هم وارد آزمون کند. سپس سناریوهای منفی مانند انقضای نشست، فایل بزرگ و شکست مدل را می‌توان به آزمون مرورگر تکرارپذیر تبدیل کرد. در همان مرحله باید احراز هویت دانلود و قدیمی‌شدن نتیجه تحلیل دارو پس از ویرایش ورودی بررسی شوند. پس از پایدارشدن این مسیرها، ارزیابی کاربردپذیری با کاربر واجد صلاحیت معنا پیدا می‌کند.

۱۱ـ۶ جمع‌بندی فنی

در این پروژه تلاش کردم سرویس‌های جدا را به یک مسیر قابل پیگیری در پرونده بیمار تبدیل کنم. کاربر ابتدا بیمار را پیدا می‌کند، هویت او را در سربرگ ثابت می‌بیند، میان داده ساختاریافته و سند جابه‌جا می‌شود و پاسخ دستیار را کنار منبع پرونده می‌خواند. در صوت و دارو نیز مرحله بازبینی از ثبت رکورد جدا مانده است. این گردش از ترکیب مسیرهای Next.js، اجزای مشترک، درخواست‌های نوع‌دار و قرارداد FastAPI ساخته شد. طبق پروپوزال، فرانت‌اند مسئولیت اصلی من بود و یکپارچه‌سازی قراردادها با همکاری عرفان انجام شد.

۱۱ـ۷ حدود نتیجه

کد، build و آزمون مرورگر نشان دادند مسیرهای اصلی رابط باز می‌شوند و گردش ساده ثبت بیمار و دارو در محیط ایزوله کار می‌کند. تصاویر صفحه عمومی، ورود، فهرست، پرونده، تب‌ها و فرم‌ها همین محدوده را مستند می‌کنند. صحت پاسخ هوش مصنوعی، پردازش فایل و اثر بالینی در این پروژه ارزیابی نشدند. پیش از استفاده جدی، مشکلات قرارداد دانلود، مدیریت نشست، وضعیت عملیات نیمه‌کاره و خطاهای lint باید برطرف شوند.

۱۱ـ۸ مسیر ادامه

گام بعدی من برای این رابط، تبدیل سناریوهای دستی به آزمون مرورگر قابل تکرار با داده ساختگی است؛ آزمون باید خطاهای ورود، ثبت، فایل و پاسخ مدل را هم پوشش دهد. پس از رفع lint و ناسازگاری‌های احراز هویت و وضعیت، می‌توان مطالعه کنترل‌شده کاربردپذیری را با کاربران واجد صلاحیت طراحی کرد. استفاده درمانی مستقل به ارزیابی‌های فنی، امنیتی و بالینی بیشتری نیاز دارد.